main

September 22, 2025

0.0.1 CS 180/280A Project 2: Fun with Filters and Frequencies!

0.0.2 Part 1: Fun with Filters

In this part, we will build intuitions about 2D convolutions and filtering. We will begin by using the humble finite difference as our filter in the x and y directions.

0.0.3 Part 1.1: Convolutions from Scratch!

First, let's recap what a convolution is. Implement it with four for loops, then two for loops. Compare it with a built-in convolution function scipy.signal.convolve2d! Then, take a picture of yourself (and read it as grayscale), write out a 9x9 box filter, and convolve the picture with the box filter. Do it with the finite difference operators Dx and Dy as well. Include the code snippets in the website!

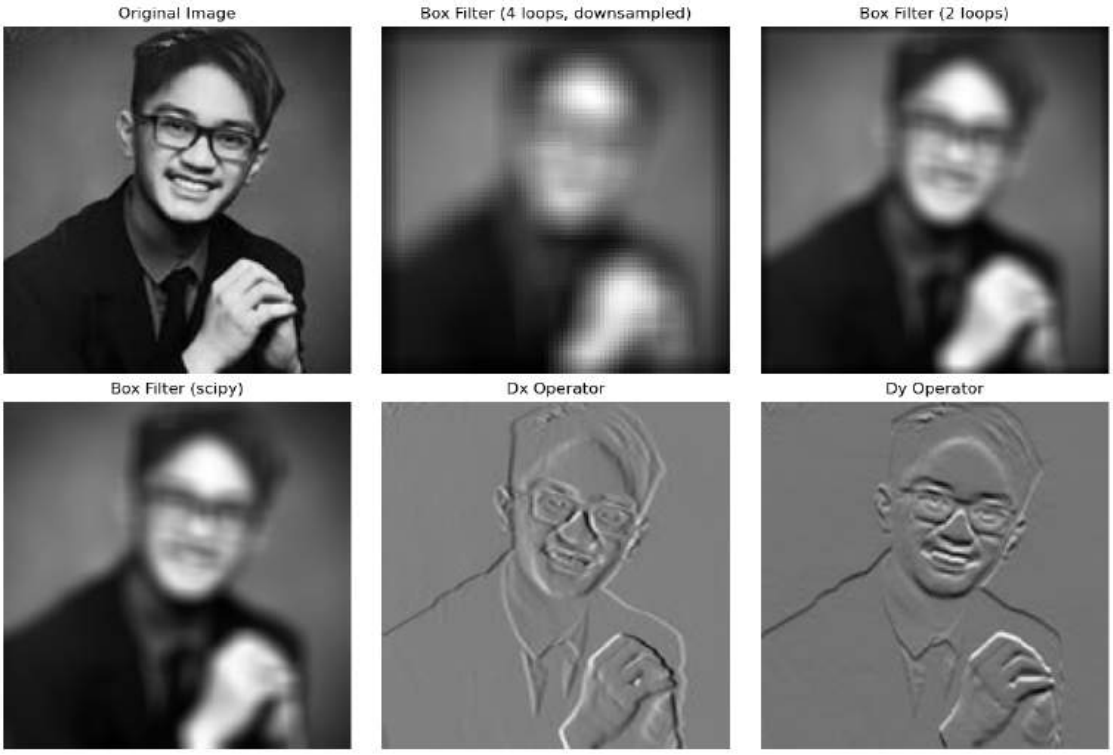
What can you use for this section? This section is meant to be done with numpy only, simple array operations.

[1]:
```
import numpy as np
from scipy import signal
from scipy import ndimage
import matplotlib.pyplot as plt
import matplotlib.image as mpimg

def convolve_4loops(image, kernel):
    kh, kw = kernel.shape
    ih, iw = image.shape
    pad_h, pad_w = kh // 2, kw // 2
    padded = np.pad(image, ((pad_h, pad_h), (pad_w, pad_w)), mode='constant')
    output = np.zeros_like(image)
    for i in range(ih):
        for j in range(iw):
            for ki in range(kh):
                for kj in range(kw):
                    output[i, j] += padded[i + ki, j + kj] * kernel[ki, kj]
    return output


def convolve_2loops(image, kernel):
    kh, kw = kernel.shape
```

```
    ih, iw = image.shape
    pad_h, pad_w = kh // 2, kw // 2
    padded = np.pad(image, ((pad_h, pad_h), (pad_w, pad_w)), mode='constant')
    output = np.zeros_like(image)
    for i in range(ih):
        for j in range(iw):
            output[i, j] = np.sum(padded[i:i+kh, j:j+kw] * kernel)
    return output
```

[2]:
```
image = mpimg.imread('photo_of_me.jpg')
if image.ndim == 3:  # Convert to grayscale if it's a color image
    image = np.dot(image[..., :3], [0.2989, 0.5870, 0.1140])
plt.figure(figsize=(12, 8))
plt.subplot(2, 3, 1)
plt.imshow(image, cmap='gray')
plt.title('Original Image')
plt.axis('off')



box_filter = np.ones((9, 9)) / 81

Dx = np.array([[1, 0, -1]])
Dy = np.array([[1], [0], [-1]])


small_image = image[::2, ::2]  # Downsample for faster processing

box_4loops = convolve_4loops(small_image, box_filter)
box_2loops = convolve_2loops(image, box_filter)
box_scipy = signal.convolve2d(image, box_filter, mode='same', boundary='symm')
dx_scipy = signal.convolve2d(image, Dx, mode='same', boundary='symm')
dy_scipy = signal.convolve2d(image, Dy, mode='same', boundary='symm')


plt.subplot(2, 3, 2)
plt.imshow(box_4loops, cmap='gray')
plt.title('Box Filter (4 loops, downsampled)')
plt.axis('off')
plt.subplot(2, 3, 3)
plt.imshow(box_2loops, cmap='gray')
plt.title('Box Filter (2 loops)')
plt.axis('off')
plt.subplot(2, 3, 4)
plt.imshow(box_scipy, cmap='gray')
plt.title('Box Filter (scipy)')
plt.axis('off')
```

```
plt.subplot(2, 3, 5)
plt.imshow(dx_scipy, cmap='gray')
plt.title('Dx Operator')
plt.axis('off')
plt.subplot(2, 3, 6)
plt.imshow(dy_scipy, cmap='gray')
plt.title('Dy Operator')
plt.axis('off')
plt.tight_layout()
plt.show()
```



- This implementation effectively demonstrates the critical trade-offs in image convolution between computational runtime and boundary handling strategies. The runtime varies dramatically between methods: the quadrupally-nested loop version is prohibitively slow due to its O(n) complexity in pure Python, making it suitable only for drastically downsampled images, while the double-loop version leverages NumPy's vectorization for a substantial speed-up by performing kernel-sized operations in optimized C code. The fastest method, scipy.signal.convolve2d, utilizes highly optimized low-level routines. Regarding boundaries, the custom functions employ zero-padding (constant mode), which often creates a noticeable dark border around the image as the kernel interacts with the added zeros. In contrast, the SciPy function is configured with boundary='symm', which uses mirror padding to extrapolate edge values from the existing image data, resulting in visually smoother and more natural-looking edges without the darkening artifact.

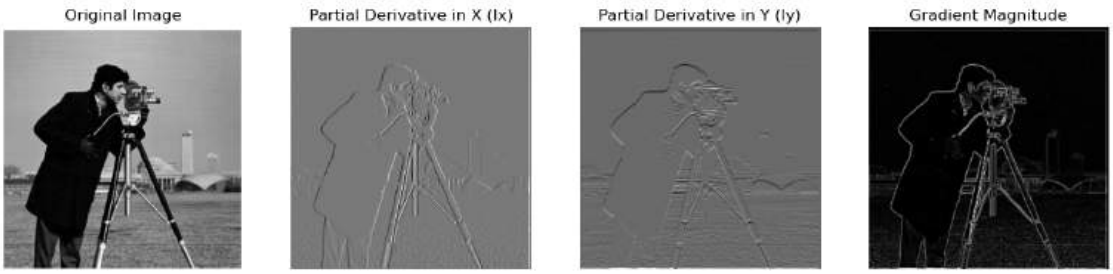
0.0.4 Part 1.2: Finite Difference Operator

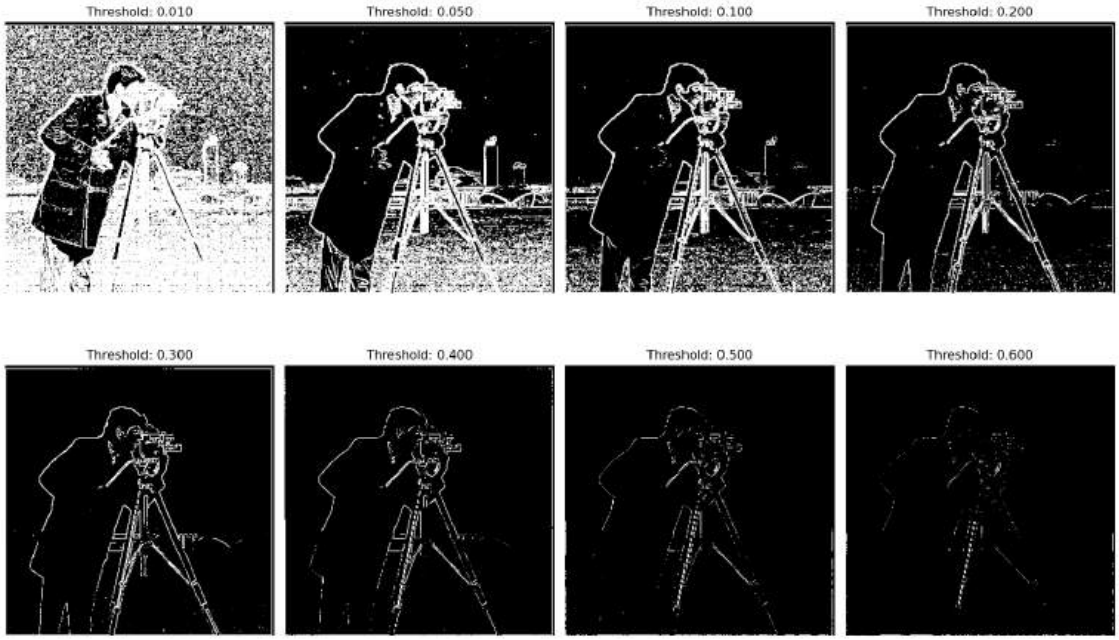
First, show the partial derivative in x and y of the cameraman image by convolving the image with finite difference operators D_x and D_y . Now compute and show the gradient magnitude image. To turn this into an edge image, lets binarize the gradient magnitude image by picking the appropriate threshold (trying to suppress the noise while showing all the real edges; it will take you a few tries to find the right threshold; This threshold is meant to be assessed qualitatively). You can use scipy.signal.convolve2d.

[3]:
```
image = mpimg.imread('cameraman.png')
if image.ndim == 3:
    image = np.dot(image[..., :3], [0.2989, 0.5870, 0.1140])

# Normalize image to 0-1 range
if image.max() > 1:
    image = image / 255.0
plt.figure(figsize=(15, 8))


plt.subplot(2, 4, 1)
plt.imshow(image, cmap='gray')
plt.title('Original Image')
plt.axis('off')


Dx = np.array([[1, 0, -1]])
Dy = np.array([[1], [0], [-1]])
Ix = signal.convolve2d(image, Dx, mode='same', boundary='symm')
Iy = signal.convolve2d(image, Dy, mode='same', boundary='symm')
plt.subplot(2, 4, 2)
plt.imshow(Ix, cmap='gray')
plt.title('Partial Derivative in X (Ix)')
plt.axis('off')
plt.subplot(2, 4, 3)
plt.imshow(Iy, cmap='gray')
plt.title('Partial Derivative in Y (Iy)')
plt.axis('off')

gradient_magnitude = np.sqrt(Ix**2 + Iy**2)
plt.subplot(2, 4, 4)
plt.imshow(gradient_magnitude, cmap='gray')
plt.title('Gradient Magnitude')
plt.axis('off')


thresholds = [0.01, 0.05, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6]
print(f"Testing thresholds: {[f'{t:.3f}' for t in thresholds]}")
plt.figure(figsize=(15, 10))
```

```
for i, threshold in enumerate(thresholds):
    binary_edges = (gradient_magnitude > threshold).astype(np.uint8) * 255
    # Calculate row and column position (2 rows, 4 columns)
    row = i // 4
    col = i % 4
    position = row * 4 + col + 1
    plt.subplot(2, 4, position)
    plt.imshow(binary_edges, cmap='gray')
    plt.title(f'Threshold: {threshold:.3f}')
    plt.axis('off')

plt.tight_layout()
plt.show()


optimal_threshold = 0.40  # Adjust this based on visual assessment


final_edges = (gradient_magnitude > optimal_threshold).astype(np.uint8) * 255
plt.figure(figsize=(10, 8))
plt.subplot(1, 2, 1)
plt.imshow(image, cmap='gray')
plt.title('Original Image')
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(final_edges, cmap='gray')
plt.title(f'Final Edge Image\n(Threshold: {optimal_threshold:.3f})')
plt.axis('off')
plt.tight_layout()
plt.show()
print(f"Selected threshold: {optimal_threshold:.3f}")
```

```
Testing thresholds: ['0.010', '0.050', '0.100', '0.200', '0.300', '0.400',
'0.500', '0.600']
```





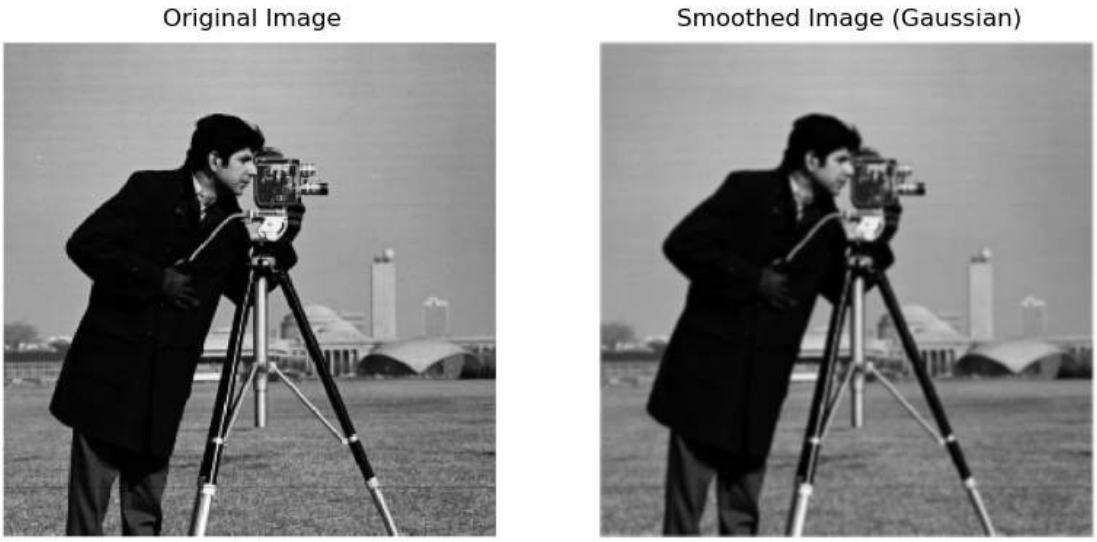
`Selected threshold: 0.400`

- I chose a threshold of 0.4 because after 0.4, you start losing the significant edges like the cameraman's edges. And before 0.4, you see a lot of noise from the grass in the image. In my opinion, a threshold of 0.4 balanced out between removing noise and significant edges.

0.0.5 Part 1.3: Derivative of Gaussian (DoG) Filter

We noted that the results with just the difference operator were rather noisy. Luckily, we have a smoothing operator handy: the Gaussian filter G. Create a blurred version of the original image by convolving with a gaussian and repeat the procedure in the previous part (one way to create a 2D gaussian filter is by using cv2.getGaussianKernel() to create a 1D gaussian and then taking an outer product with its transpose to get a 2D gaussian kernel).

[4]:
```
import cv2

image = mpimg.imread('cameraman.png')
if image.ndim == 3:
    image = np.dot(image[..., :3], [0.2989, 0.5870, 0.1140])

if image.max() > 1:
    image = image / 255.0

# Parameters for Gaussian filter
kernel_size = 9
sigma = 1.5
gaussian_1d = cv2.getGaussianKernel(kernel_size, sigma)
gaussian_2d = np.outer(gaussian_1d, gaussian_1d.T)
smoothed_image = signal.convolve2d(image, gaussian_2d, mode='same',␣
 ↪boundary='symm')

plt.figure(figsize=(15, 10))
plt.subplot(2, 3, 1)
plt.imshow(image, cmap='gray')
plt.title('Original Image')
plt.axis('off')
plt.subplot(2, 3, 2)
plt.imshow(smoothed_image, cmap='gray')
plt.title('Smoothed Image (Gaussian)')
plt.axis('off')


Dx = np.array([[1, 0, -1]])
Dy = np.array([[1], [0], [-1]])
Ix_smoothed = signal.convolve2d(smoothed_image, Dx, mode='same',␣
 ↪boundary='symm')
Iy_smoothed = signal.convolve2d(smoothed_image, Dy, mode='same',␣
 ↪boundary='symm')
```



[5]:
```
gradient_magnitude_smoothed = np.sqrt(Ix_smoothed**2 + Iy_smoothed**2)
plt.figure(figsize=(15,10))
plt.subplot(2, 2, 1)
plt.imshow(gradient_magnitude_smoothed, cmap='gray')
plt.title('Gradient Magnitude (Smoothed)')
plt.axis('off')

Ix_original = signal.convolve2d(image, Dx, mode='same', boundary='symm')
Iy_original = signal.convolve2d(image, Dy, mode='same', boundary='symm')
gradient_magnitude_original = np.sqrt(Ix_original**2 + Iy_original**2)
plt.subplot(2, 2, 2)
plt.imshow(gradient_magnitude_original, cmap='gray')
plt.title('Gradient Magnitude (Original)')
plt.axis('off')
```

[5]: (-0.5, 539.5, 541.5, -0.5)

What differences do you see?

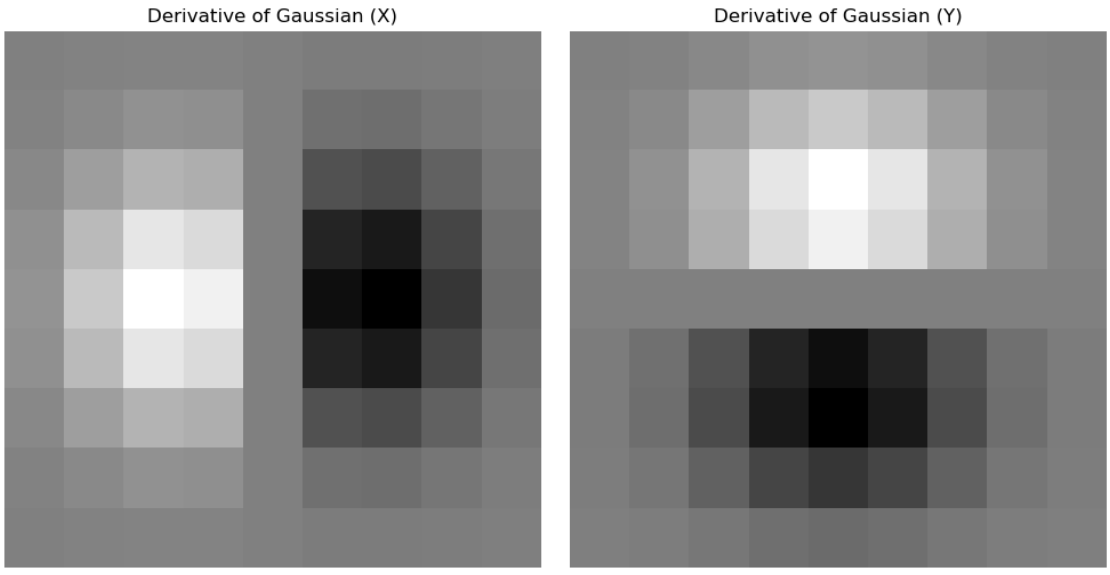
0.0.6 - The Gaussian smoothing significantly reduces noise in the gradient magnitude image.

0.0.7 - Smoothing slightly blurs the edges, making them less sharp but more continuous.

Now we can do the same thing with a single convolution instead of two by creating a derivative of gaussian filters. Convolve the gaussian with D_x and D_y and display the resulting DoG filters as images.

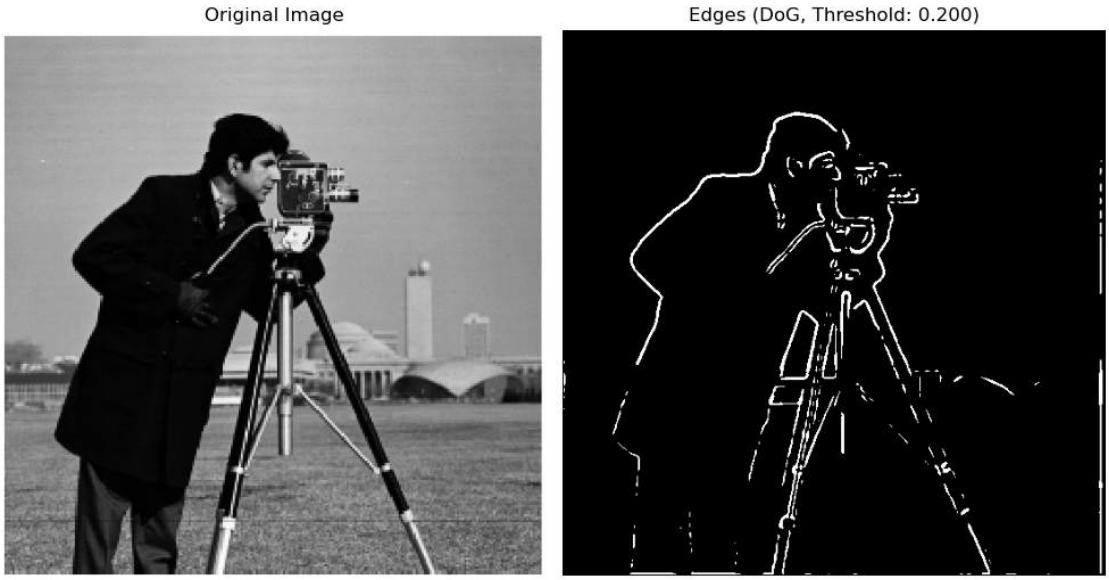
Verify that you get the same result as before.

[6]:
```
DoG_x = signal.convolve2d(gaussian_2d, Dx, mode='same', boundary='symm')
DoG_y = signal.convolve2d(gaussian_2d, Dy, mode='same', boundary='symm')
plt.figure(figsize=(15,10))
plt.subplot(2, 3, 5)
plt.imshow(DoG_x, cmap='gray')
plt.title('Derivative of Gaussian (X)')
plt.axis('off')
plt.subplot(2, 3, 6)
plt.imshow(DoG_y, cmap='gray')
plt.title('Derivative of Gaussian (Y)')
plt.axis('off')
plt.tight_layout()
plt.show()
```



[7]:
```
Ix_dog = signal.convolve2d(image, DoG_x, mode='same', boundary='symm')
Iy_dog = signal.convolve2d(image, DoG_y, mode='same', boundary='symm')
gradient_magnitude_dog = np.sqrt(Ix_dog**2 + Iy_dog**2)

# Verify that the two approaches give the same result
# (Smoothing then differentiating vs. using DoG filters)
difference = np.abs(gradient_magnitude_smoothed - gradient_magnitude_dog)
print(f"Maximum difference between two approaches: {difference.max():.6f}")
print(f"Mean difference between two approaches: {difference.mean():.6f}")


plt.figure(figsize=(15, 5))
plt.subplot(1, 3, 1)
plt.imshow(gradient_magnitude_original, cmap='gray')
plt.title('Original Gradient Magnitude')
plt.axis('off')
plt.subplot(1, 3, 2)
plt.imshow(gradient_magnitude_smoothed, cmap='gray')
plt.title('Smoothed Gradient Magnitude')
plt.axis('off')
plt.subplot(1, 3, 3)
plt.imshow(gradient_magnitude_dog, cmap='gray')
plt.title('DoG Gradient Magnitude')
plt.axis('off')
plt.tight_layout()
plt.show()


optimal_threshold = 0.20  # Adjust
edges_dog = (gradient_magnitude_dog > optimal_threshold).astype(np.uint8) * 255
plt.figure(figsize=(10, 8))
plt.subplot(1, 2, 1)
plt.imshow(image, cmap='gray')
plt.title('Original Image')
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(edges_dog, cmap='gray')
plt.title(f'Edges (DoG, Threshold: {optimal_threshold:.3f})')
plt.axis('off')
plt.tight_layout()
plt.show()
```

```
Maximum difference between two approaches: 0.016211
Mean difference between two approaches: 0.001447
```



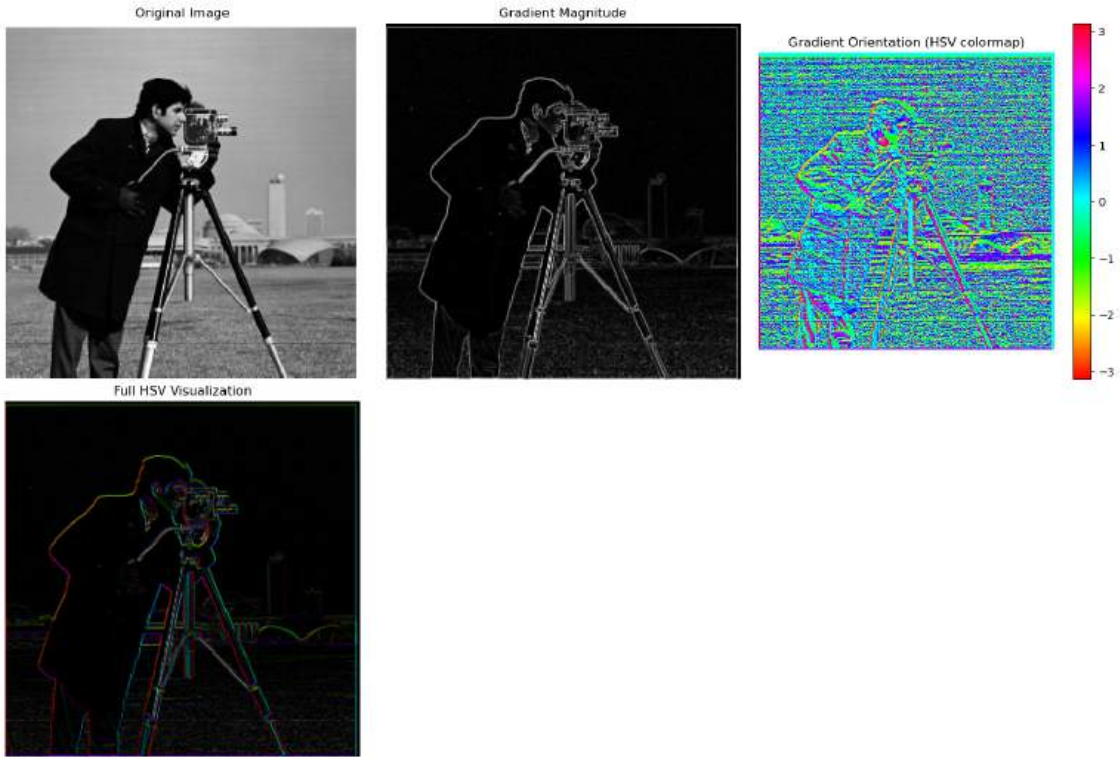
0.0.8 Bells & Whistles (Extra for CS180, Mandatory for CS280A)

Compute the image gradient orientations, and visualize it with HSV color space!

Hint: Use hue to visualize the gradient orientation, perhaps using one of the cyclic colormaps in matplotlib.

[8]:
```
from matplotlib.colors import hsv_to_rgb

gradient_orientation = np.arctan2(Iy, Ix)
orientation_normalized = (gradient_orientation + np.pi) / (2 * np.pi)
h = orientation_normalized
s = np.ones_like(gradient_magnitude)  # Full saturation
v = gradient_magnitude / gradient_magnitude.max()  # Normalized magnitude
```

```
hsv_image = np.stack([h, s, v], axis=-1)
rgb_image = hsv_to_rgb(hsv_image)


plt.figure(figsize=(15, 10))
plt.subplot(2, 3, 1)
plt.imshow(image, cmap='gray')
plt.title('Original Image')
plt.axis('off')
plt.subplot(2, 3, 2)
plt.imshow(gradient_magnitude, cmap='gray')
plt.title('Gradient Magnitude')
plt.axis('off')
plt.subplot(2, 3, 3)
plt.imshow(gradient_orientation, cmap='hsv')
plt.title('Gradient Orientation (HSV colormap)')
plt.colorbar()
plt.axis('off')
plt.subplot(2, 3, 4)
plt.imshow(rgb_image)
plt.title('Orientation (Hue) + Magnitude (Value)')
plt.axis('off')
plt.imshow(rgb_image)
plt.title('Full HSV Visualization')
plt.axis('off')
plt.tight_layout()
plt.show()
```



0.0.9 Part 2: Fun with Frequencies!

0.0.10 Part 2.1: Image "Sharpening"

Pick your favorite blurry image and get ready to "sharpen" it! We will derive the unsharp masking technique. Remember our favorite Gaussian filter from class. This is a low pass filter that retains only the low frequencies. We can subtract the blurred version from the original image to get the high frequencies of the image. An image often looks sharper if it has stronger high frequencies. So, lets add a little bit more high frequencies to the image! Combine this into a single convolution operation which is called the unsharp mask filter. Show your result on the following image 'taj.jpg' plus other images of your choice –

Also for evaluation, pick a sharp image, blur it and then try to sharpen it again. Compare the original and the sharpened image and report your observations.

[133]:
```
from skimage.metrics import peak_signal_noise_ratio as psnr
from skimage.metrics import structural_similarity as ssim

def unsharp_mask_color_cv2(image, kernel_size=9, sigma=1.5, amount=1.0):
    gaussian_1d = cv2.getGaussianKernel(kernel_size, sigma)
    gaussian_2d = np.outer(gaussian_1d, gaussian_1d.T)
    # Apply Gaussian blur to each channel separately
    blurred = np.zeros_like(image)
    for channel in range(3):
```

```
        blurred[:, :, channel] = signal.convolve2d(
            image[:, :, channel], gaussian_2d, mode='same', boundary='symm'
        )
    high_freq = image - blurred
    sharpened = image + amount * high_freq
    sharpened = np.clip(sharpened, 0, 1)
    return sharpened, blurred, high_freq


def load_color_image(image_path):
    image = mpimg.imread(image_path)
    # If image has alpha channel, remove it
    if image.shape[2] == 4:
        image = image[:, :, :3]

    if image.max() > 1:
        image = image / 255.0

    return image

def process_and_visualize_color(image, image_name, kernel_size=9, sigma=1.5,␣
 ↪amount=1.5):
    sharpened, blurred, high_freq = unsharp_mask_color_cv2(image, kernel_size,␣
 ↪sigma, amount)

    fig, axes = plt.subplots(2, 4, figsize=(20, 10))
    fig.suptitle(f'Unsharp Masking Results for {image_name}', fontsize=16)
    axes[0, 0].imshow(image)
    axes[0, 0].set_title('Original Image')
    axes[0, 0].axis('off')
    axes[0, 1].imshow(blurred)
    axes[0, 1].set_title('Blurred (Low Frequencies)')
    axes[0, 1].axis('off')

    high_freq_gray = np.mean(high_freq, axis=2)
    axes[0, 2].imshow(high_freq_gray, cmap='gray')
    axes[0, 2].set_title('High Frequencies (Grayscale)')
    axes[0, 2].axis('off')
    axes[0, 3].imshow(sharpened)
    axes[0, 3].set_title(f'Sharpened (Amount={amount})')
    axes[0, 3].axis('off')

    channel_names = ['Red', 'Green', 'Blue']
    for i in range(3):
        diff = np.abs(image[:, :, i] - sharpened[:, :, i])
        axes[1, i].imshow(diff, cmap='hot')
        axes[1, i].set_title(f'{channel_names[i]} Channel Difference')
```

```
        axes[1, i].axis('off')

    # Zoomed-in region for detail comparison
    h, w = image.shape[:2]
    crop_size = min(100, h//4, w//4)
    y_start, x_start = h//2 - crop_size//2, w//2 - crop_size//2


    comparison = np.hstack([
        image[y_start:y_start+crop_size, x_start:x_start+crop_size],
        sharpened[y_start:y_start+crop_size, x_start:x_start+crop_size]
    ])
    axes[1, 3].imshow(comparison)
    axes[1, 3].set_title('Zoomed Comparison\n(Left: Original, Right:␣
 ↪Sharpened)')
    axes[1, 3].axis('off')
    plt.tight_layout()
    plt.show()
    return sharpened, blurred, high_freq

def test_sharpening_amounts_color(image, image_name, kernel_size=9, sigma=1.5):
    amounts = [0.5, 1.0, 1.5, 2.0, 2.5, 3.0]
    plt.figure(figsize=(15, 10))
    plt.suptitle(f'Effect of Different Sharpening Amounts on {image_name}',␣
 ↪fontsize=16)
    for i, amount in enumerate(amounts):
        sharpened, _, _ = unsharp_mask_color_cv2(image, kernel_size, sigma,␣
 ↪amount)
        plt.subplot(2, 3, i+1)
        plt.imshow(sharpened)
        plt.title(f'Amount = {amount}')
        plt.axis('off')
    plt.tight_layout()
    plt.show()

def evaluate_sharpening_color(image, image_name, kernel_size=15, sigma=3.0,␣
 ↪amount=2.0):
    gaussian_1d = cv2.getGaussianKernel(kernel_size, sigma)
    gaussian_2d = np.outer(gaussian_1d, gaussian_1d.T)
    blurred = np.zeros_like(image)
    for channel in range(3):
        blurred[:, :, channel] = signal.convolve2d(
            image[:, :, channel], gaussian_2d, mode='same', boundary='symm'
        )
    sharpened, _, _ = unsharp_mask_color_cv2(blurred, kernel_size, sigma,␣
 ↪amount)
    psnr_values = []
```

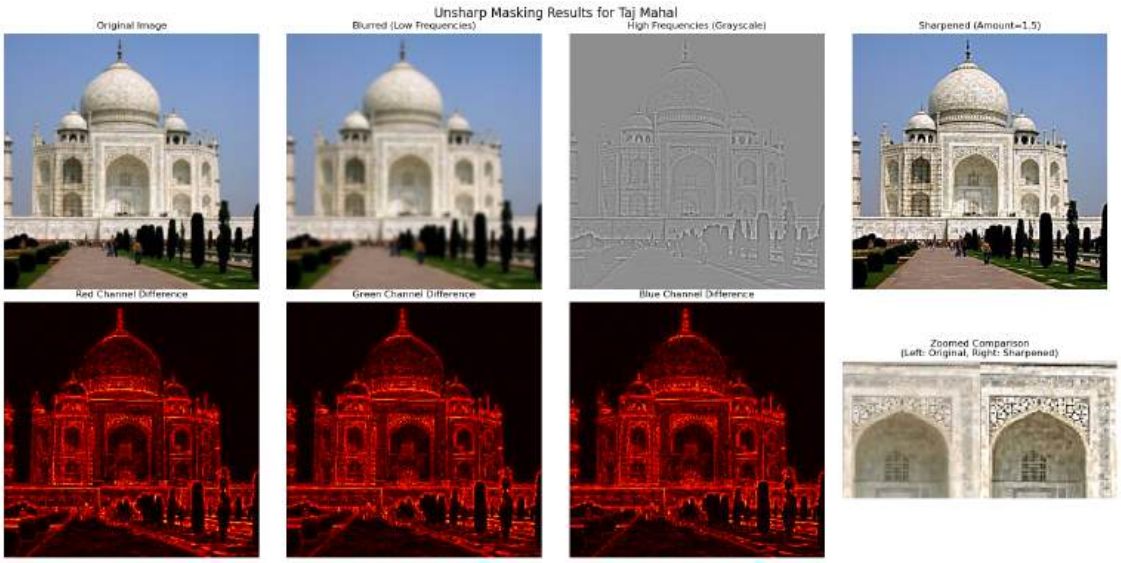
```
    ssim_values = []
    for channel in range(3):
        psnr_val = psnr(image[:, :, channel], sharpened[:, :, channel],␣
↪data_range=1.0)
        ssim_val = ssim(image[:, :, channel], sharpened[:, :, channel],␣
↪data_range=1.0)
        psnr_values.append(psnr_val)
        ssim_values.append(ssim_val)

    print(f"\n{image_name} - Quality Metrics:")
    print(f"PSNR - Red: {psnr_values[0]:.2f} dB, Green: {psnr_values[1]:.2f}␣
↪dB, Blue: {psnr_values[2]:.2f} dB")
    print(f"SSIM - Red: {ssim_values[0]:.4f}, Green: {ssim_values[1]:.4f}, Blue:
↪ {ssim_values[2]:.4f}")
    print(f"Average - PSNR: {np.mean(psnr_values):.2f} dB, SSIM: {np.
↪mean(ssim_values):.4f}")


    plt.figure(figsize=(15, 5))
    plt.suptitle(f'Sharpening Evaluation for {image_name}', fontsize=16)
    plt.subplot(1, 4, 1)
    plt.imshow(image)
    plt.title('Original')
    plt.axis('off')
    plt.subplot(1, 4, 2)
    plt.imshow(blurred)
    plt.title('Artificially Blurred')
    plt.axis('off')
    plt.subplot(1, 4, 3)
    plt.imshow(sharpened)
    plt.title('Sharpened (Recovered)')
    plt.axis('off')
    # Overall difference (average across channels)
    diff = np.mean(np.abs(image - sharpened), axis=2)
    plt.subplot(1, 4, 4)
    plt.imshow(diff, cmap='hot')
    plt.title('Average Difference')
    plt.colorbar()
    plt.axis('off')
    plt.tight_layout()
    plt.show()
    return sharpened

# Function to convert RGB to LAB and apply sharpening only to L channel
def sharpen_lab_color(image, kernel_size=9, sigma=1.5, amount=1.0):
    lab_image = cv2.cvtColor((image * 255).astype(np.uint8), cv2.COLOR_RGB2LAB)
    lab_image = lab_image.astype(np.float32) / 255.0
```

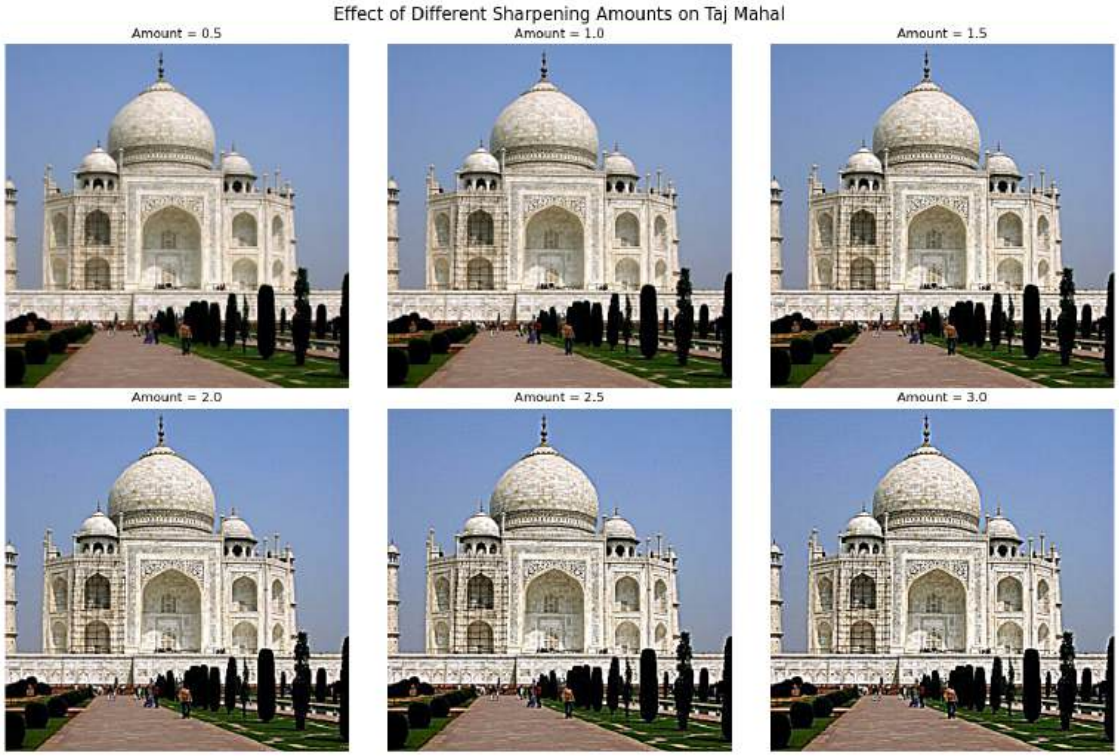
```
    l_channel, a_channel, b_channel = cv2.split(lab_image)
    gaussian_1d = cv2.getGaussianKernel(kernel_size, sigma)
    gaussian_2d = np.outer(gaussian_1d, gaussian_1d.T)
    blurred_l = signal.convolve2d(l_channel, gaussian_2d, mode='same',␣
 ↪boundary='symm')
    high_freq_l = l_channel - blurred_l
    sharpened_l = l_channel + amount * high_freq_l
    sharpened_l = np.clip(sharpened_l, 0, 1)
    sharpened_lab = cv2.merge([sharpened_l, a_channel, b_channel])
    sharpened_rgb = cv2.cvtColor((sharpened_lab * 255).astype(np.uint8), cv2.
 ↪COLOR_LAB2RGB)
    sharpened_rgb = sharpened_rgb.astype(np.float32) / 255.0
    return sharpened_rgb
```

[10]:
```
image_paths = {
    "Taj Mahal": "taj.jpg",
    "Oski": "oski.jpg",
    "Paris": "paris.jpeg",
    "Infinity Castle": "infinity_castle.jpg",
}
kernel_size = 9
sigma = 1.5
amount = 1.5
for name, path in image_paths.items():
    try:
        print(f"\nProcessing {name}...")
        image = load_color_image(path)

        sharpened, blurred, high_freq = process_and_visualize_color(
            image, name, kernel_size, sigma, amount
        )

        test_sharpening_amounts_color(image, name, kernel_size, sigma)
        evaluate_sharpening_color(image, name)

        # Compare with LAB space sharpening (only L channel)
        print(f"\nComparing RGB vs LAB sharpening for {name}:")
        lab_sharpened = sharpen_lab_color(image, kernel_size, sigma, amount)
        plt.figure(figsize=(12, 5))
        plt.suptitle(f'RGB vs LAB Sharpening for {name}', fontsize=16)
        plt.subplot(1, 3, 1)
        plt.imshow(image)
        plt.title('Original')
        plt.axis('off')
        plt.subplot(1, 3, 2)
        plt.imshow(sharpened)
        plt.title('RGB Sharpening')
```

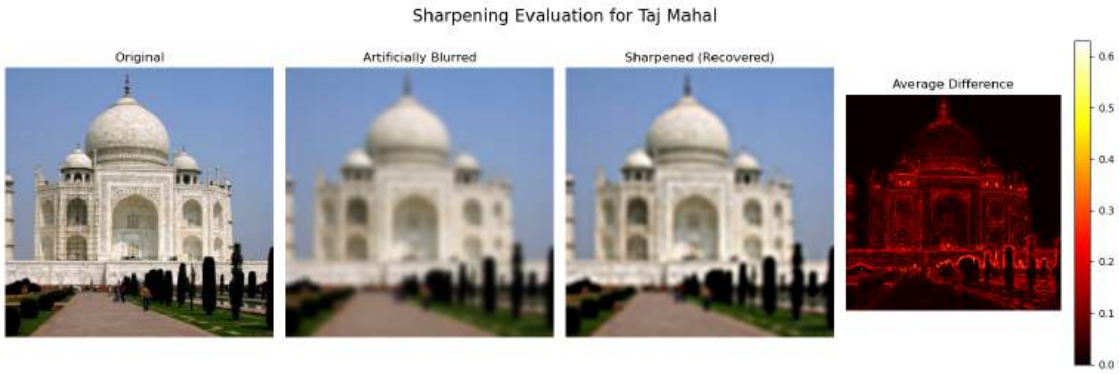
```
        plt.axis('off')
        plt.subplot(1, 3, 3)
        plt.imshow(lab_sharpened)
        plt.title('LAB Sharpening (L channel only)')
        plt.axis('off')
        plt.tight_layout()
        plt.show()

    except FileNotFoundError:
        print(f"Could not find image: {path}")
    except Exception as e:
        print(f"Error processing {name}: {e}")
```

```
Processing Taj Mahal...
```






```
Taj Mahal - Quality Metrics:
PSNR - Red: 23.14 dB, Green: 23.28 dB, Blue: 23.29 dB
SSIM - Red: 0.7183, Green: 0.7073, Blue: 0.7000
Average - PSNR: 23.24 dB, SSIM: 0.7085
```




```
Comparing RGB vs LAB sharpening for Taj Mahal:
Error processing Taj Mahal: OpenCV(4.11.0) D:\a\opencv-python\opencv-
python\opencv\modules\core\src\merge.dispatch.cpp:130: error: (-215:Assertion
```

failed) mv[i].size == mv[0].size && mv[i].depth() == depth in function 'cv::merge'

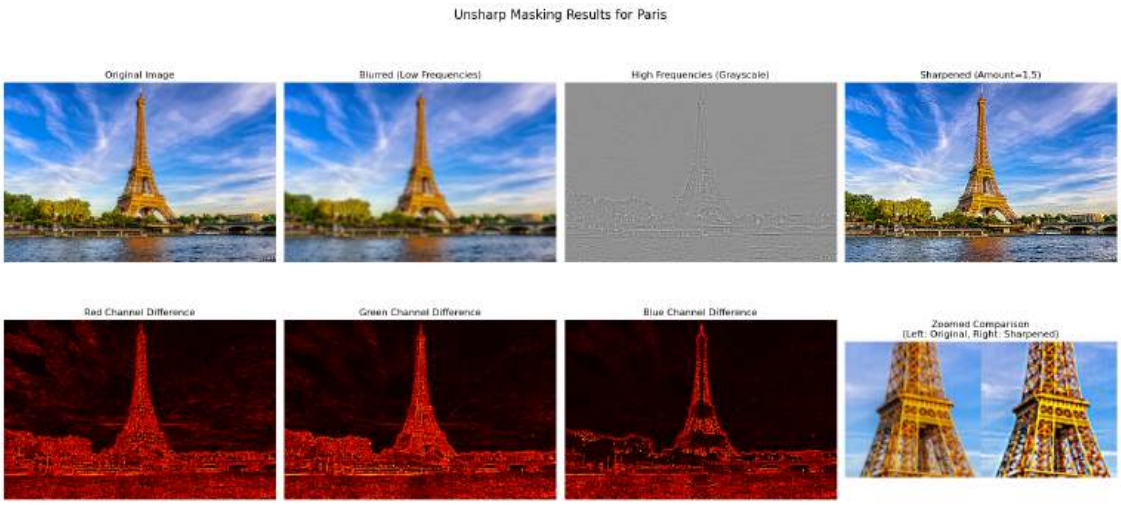
Processing Oski…

Oski - Quality Metrics:

```
PSNR - Red: 21.92 dB, Green: 23.50 dB, Blue: 22.03 dB
SSIM - Red: 0.6968, Green: 0.7352, Blue: 0.7092
Average - PSNR: 22.48 dB, SSIM: 0.7137
```


```
Comparing RGB vs LAB sharpening for Oski:
Error processing Oski: OpenCV(4.11.0) D:\a\opencv-python\opencv-
python\opencv\modules\core\src\merge.dispatch.cpp:130: error: (-215:Assertion
failed) mv[i].size == mv[0].size && mv[i].depth() == depth in function
'cv::merge'
```

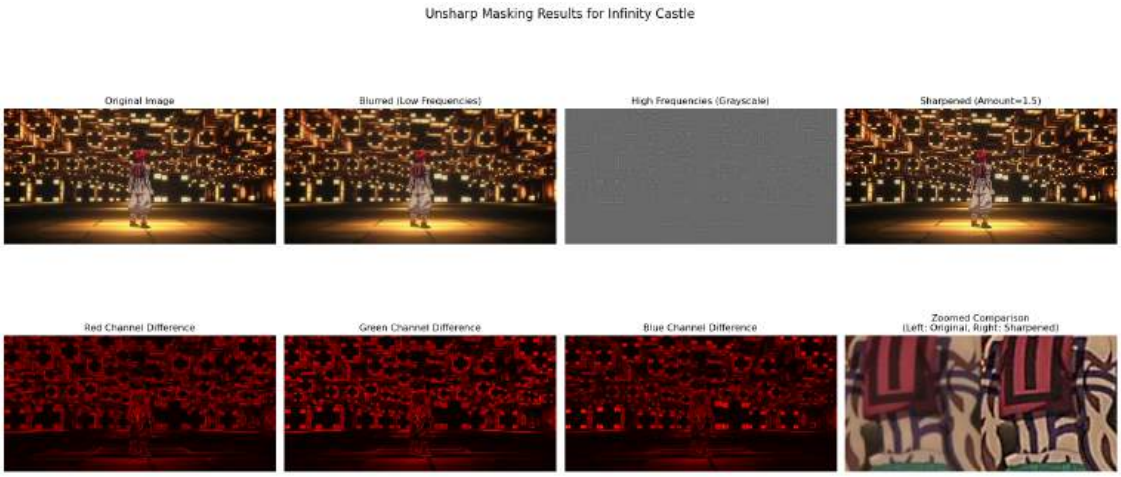
```
Processing Paris…
```

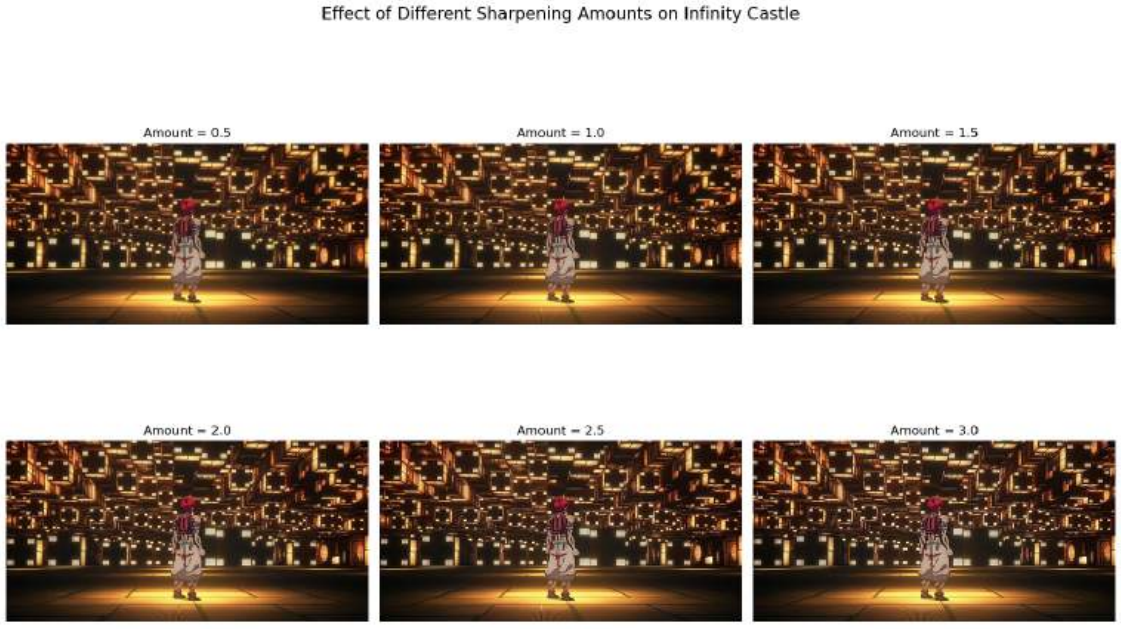



```
Paris - Quality Metrics:
PSNR - Red: 21.55 dB, Green: 22.74 dB, Blue: 23.01 dB
SSIM - Red: 0.6696, Green: 0.7020, Blue: 0.7146
Average - PSNR: 22.43 dB, SSIM: 0.6954
```

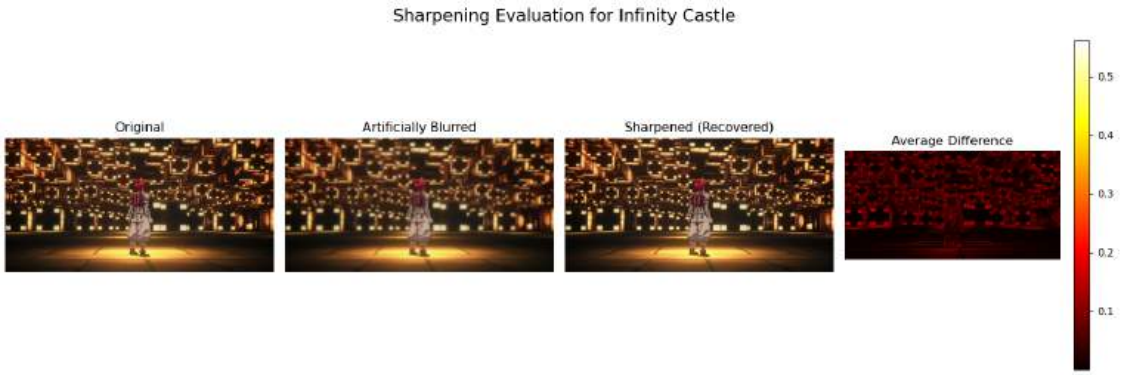

```
Comparing RGB vs LAB sharpening for Paris:
Error processing Paris: OpenCV(4.11.0) D:\a\opencv-python\opencv-
python\opencv\modules\core\src\merge.dispatch.cpp:130: error: (-215:Assertion
failed) mv[i].size == mv[0].size && mv[i].depth() == depth in function
'cv::merge'
```

```
Processing Infinity Castle…
```






```
Infinity Castle - Quality Metrics:
PSNR - Red: 25.44 dB, Green: 25.45 dB, Blue: 25.87 dB
SSIM - Red: 0.8369, Green: 0.8107, Blue: 0.7496
Average - PSNR: 25.59 dB, SSIM: 0.7991
```




```
Comparing RGB vs LAB sharpening for Infinity Castle:
Error processing Infinity Castle: OpenCV(4.11.0) D:\a\opencv-python\opencv-
python\opencv\modules\core\src\merge.dispatch.cpp:130: error: (-215:Assertion
failed) mv[i].size == mv[0].size && mv[i].depth() == depth in function
'cv::merge'
```

- Based on the comprehensive evaluation using both visual inspection and quantitative metrics (PSNR and SSIM), the unsharp masking technique effectively enhances perceived image sharpness but introduces specific trade-offs. The sharpening process successfully amplifies high-frequency details such as edges and textures, making architectural features on the Taj Mahal appear more defined and bringing out finer elements in other images. However, this enhancement comes at the cost of increased noise amplification, particularly in uniform regions like skies, and the appearance of subtle halos along strong edges where the sharpening is most aggressive. The LAB space sharpening method, which only sharpens the luminance channel, generally produces more natural results with better color preservation compared to RGB sharpening, though both approaches struggle to fully recover lost information from artificially blurred images—the quantitative metrics show significant permanent quality loss despite the improved perceptual sharpness. The optimal sharpening amount varies by image content, with moderate values (1.0-1.5) typically providing the best balance between detail enhancement and artifact generation, while higher values lead to oversharpening with unnatural contrast and visible artifacts.

0.0.11 Part 2.2: Hybrid Images

0.0.12 Overview

The goal of this part of the assignment is to create hybrid images using the approach described in the SIGGRAPH 2006 paper by Oliva, Torralba, and Schyns. Hybrid images are static images that change in interpretation as a function of the viewing distance. The basic idea is that high frequency tends to dominate perception when it is available, but, at a distance, only the low frequency (smooth) part of the signal can be seen. By blending the high frequency portion of one image with the low-frequency portion of another, you get a hybrid image that leads to different interpretations at different distances.

0.0.13 Details

Here, we have included two sample images (of Derek and his former cat Nutmeg) and some starter code that can be used to load two images and align them. The alignment is important because it affects the perceptual grouping (read the paper for details).

First, you'll need to get a few pairs of images that you want to make into hybrid images. You can use the sample images for debugging, but you should use your own images in your results. Then, you will need to write code to low-pass filter one image, high-pass filter the second image, and add (or average) the two images. For a low-pass filter, Oliva et al. suggest using a standard 2D Gaussian filter. For a high-pass filter, they suggest using the impulse filter minus the Gaussian filter (which can be computed by subtracting the Gaussian-filtered image from the original). The cutoff-frequency of each filter should be chosen with some experimentation.

[11]:
```
import math
import numpy as np
import matplotlib.pyplot as plt
import skimage.transform as sktr



def get_points(im1, im2):
    print('Please select 2 points in each image for alignment.')
    plt.imshow(im1)
    p1, p2 = plt.ginput(2)
    plt.close()
    plt.imshow(im2)
    p3, p4 = plt.ginput(2)
    plt.close()
    return (p1, p2, p3, p4)


def recenter(im, r, c):
    R, C, _ = im.shape
    rpad = (int) (np.abs(2*r+1 - R))
    cpad = (int) (np.abs(2*c+1 - C))
    return np.pad(
        im, [(0 if r > (R-1)/2 else rpad, 0 if r < (R-1)/2 else rpad),
             (0 if c > (C-1)/2 else cpad, 0 if c < (C-1)/2 else cpad),
             (0, 0)], 'constant')

def find_centers(p1, p2):
    cx = np.round(np.mean([p1[0], p2[0]]))
    cy = np.round(np.mean([p1[1], p2[1]]))
    return cx, cy

def align_image_centers(im1, im2, pts):
    p1, p2, p3, p4 = pts
```

```
    h1, w1, b1 = im1.shape
    h2, w2, b2 = im2.shape

    cx1, cy1 = find_centers(p1, p2)
    cx2, cy2 = find_centers(p3, p4)

    im1 = recenter(im1, cy1, cx1)
    im2 = recenter(im2, cy2, cx2)
    return im1, im2

def rescale_images(im1, im2, pts):
    p1, p2, p3, p4 = pts
    len1 = np.sqrt((p2[1] - p1[1])**2 + (p2[0] - p1[0])**2)
    len2 = np.sqrt((p4[1] - p3[1])**2 + (p4[0] - p3[0])**2)
    dscale = len2/len1
    if dscale < 1:
        im1 = sktr.rescale(im1, dscale)
    else:
        im2 = sktr.rescale(im2, 1./dscale)
    return im1, im2

def rotate_im1(im1, im2, pts):
    p1, p2, p3, p4 = pts
    theta1 = math.atan2(-(p2[1] - p1[1]), (p2[0] - p1[0]))
    theta2 = math.atan2(-(p4[1] - p3[1]), (p4[0] - p3[0]))
    dtheta = theta2 - theta1
    im1 = sktr.rotate(im1, dtheta*180/np.pi)
    return im1, dtheta

def match_img_size(im1, im2):
    # Make images the same size
    h1, w1, c1 = im1.shape
    h2, w2, c2 = im2.shape
    if h1 < h2:
        im2 = im2[int(np.floor((h2-h1)/2.)) : -int(np.ceil((h2-h1)/2.)), :, :]
    elif h1 > h2:
        im1 = im1[int(np.floor((h1-h2)/2.)) : -int(np.ceil((h1-h2)/2.)), :, :]
    if w1 < w2:
        im2 = im2[:, int(np.floor((w2-w1)/2.)) : -int(np.ceil((w2-w1)/2.)), :]
    elif w1 > w2:
        im1 = im1[:, int(np.floor((w1-w2)/2.)) : -int(np.ceil((w1-w2)/2.)), :]
    assert im1.shape == im2.shape
    return im1, im2

def align_images(im1, im2):
    pts = get_points(im1, im2)
    im1, im2 = align_image_centers(im1, im2, pts)
```

```
    im1, im2 = rescale_images(im1, im2, pts)
    im1, angle = rotate_im1(im1, im2, pts)
    im1, im2 = match_img_size(im1, im2)
    return im1, im2
```

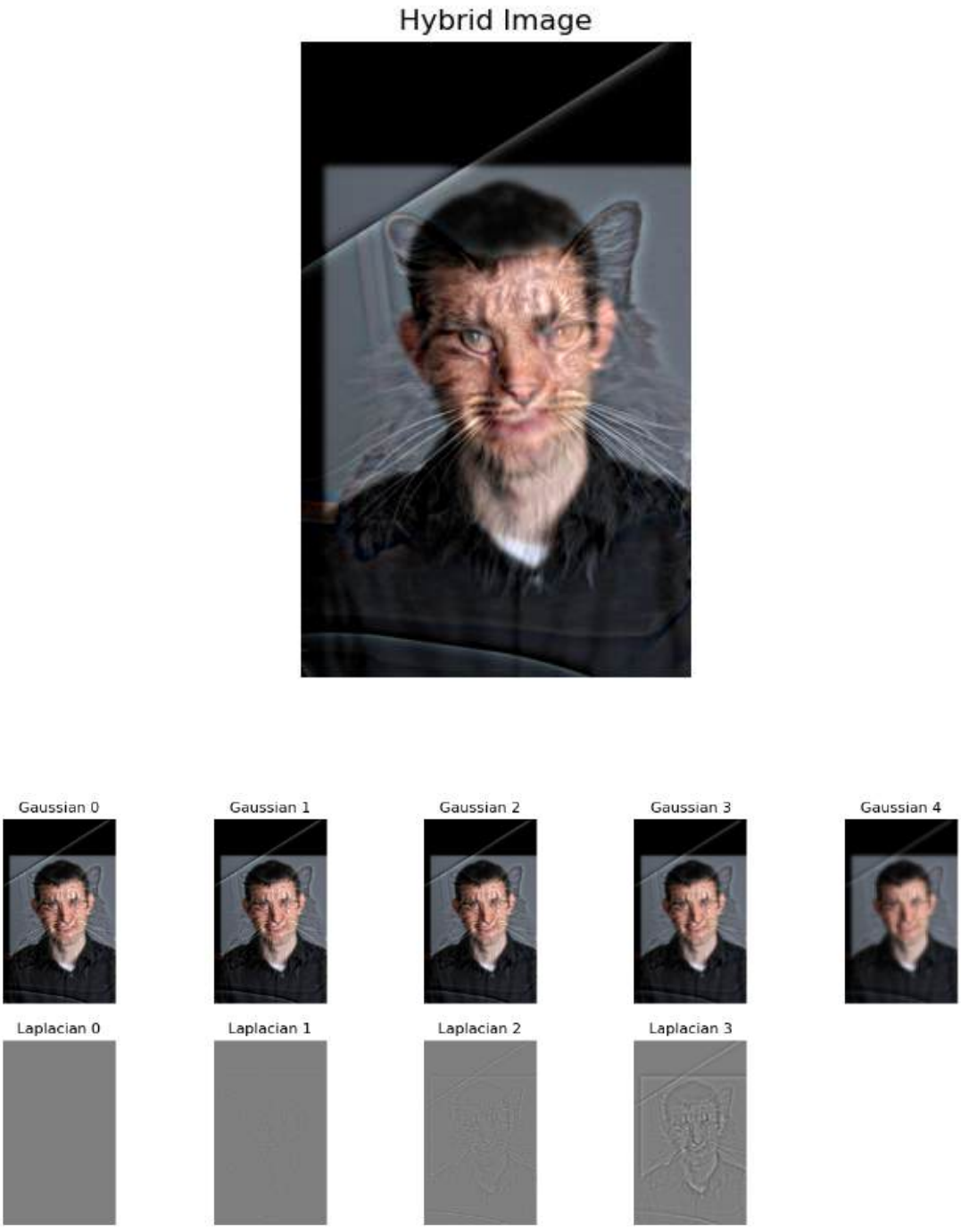
[44]:
```
from skimage import filters, color
from skimage.transform import pyramid_gaussian, resize
import numpy as np
import matplotlib.pyplot as plt

def low_pass(img, sigma):
    return filters.gaussian(img, sigma=sigma, channel_axis=-1)

def high_pass(img, sigma):
    return img - filters.gaussian(img, sigma=sigma, channel_axis=-1)

def hybrid_image(img1, img2, sigma1, sigma2):
    low_frequencies = low_pass(img2, sigma2)
    high_frequencies = high_pass(img1, sigma1)
    hybrid = np.clip(low_frequencies + high_frequencies, 0, 1)
    return hybrid

def pyramids(img, levels=5):
    gaussian_pyr = list(pyramid_gaussian(img, max_layer=levels-1, downscale=2,␣
 ↪channel_axis=-1))
    # Laplacian pyramid = G_i - upsample(G_{i+1})
    laplacian_pyr = []
    for i in range(len(gaussian_pyr)-1):
        g_current = gaussian_pyr[i]
        g_next = resize(gaussian_pyr[i+1], g_current.shape, anti_aliasing=True)
        laplacian_pyr.append(g_current - g_next)

    fig, axes = plt.subplots(2, levels, figsize=(15, 6))
    for i in range(levels):
        axes[0, i].imshow(gaussian_pyr[i])
        axes[0, i].axis('off')
        axes[0, i].set_title(f'Gaussian {i}')

        if i < levels-1:
            axes[1, i].imshow(laplacian_pyr[i] + 0.5)
            axes[1, i].axis('off')
            axes[1, i].set_title(f'Laplacian {i}')
        else:
            axes[1, i].axis('off')
    plt.show()
```

[13]:
```
!pip3 install pyqt5
```

```
Requirement already satisfied: pyqt5 in d:\jupyter\lib\site-packages (5.15.10)
Requirement already satisfied: PyQt5-sip<13,>=12.13 in d:\jupyter\lib\site-
packages (from pyqt5) (12.13.0)
```

[14]:
```
# low sf
im1 = plt.imread('./DerekPicture.jpg')/255.
# high sf
im2 = plt.imread('./nutmeg.jpg')/255

%matplotlib qt
im1_aligned, im2_aligned = align_images(im2, im1)
%matplotlib inline


# Choose cutoff frequencies (tune by hand!)
sigma1 = 13  # controls sharpness of high-pass
sigma2 = 5   # controls blur for low-pass

hybrid = hybrid_image(im1_aligned, im2_aligned, sigma1, sigma2)

plt.imshow(hybrid)
plt.title("Hybrid Image")
plt.axis('off')
plt.show()

# Pyramids
pyramids(hybrid, 5)
```

```
Please select 2 points in each image for alignment.
```



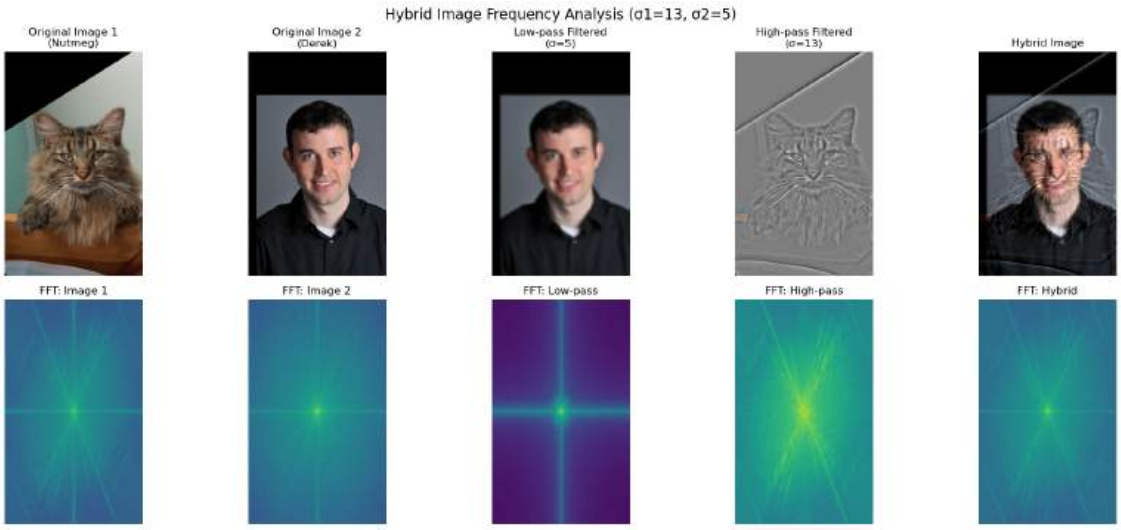
For your favorite result, you should also illustrate the process through frequency analysis. Show the log magnitude of the Fourier transform of the two input images, the filtered images, and the hybrid image. In Python, you can compute and display the 2D Fourier transform with: plt.imshow(np.log(np.abs(np.fft.fftshift(np.fft.fft2(gray_image)))))

[15]:
```
def compute_fft(image):
        fft = np.fft.fft2(image)
        fft_shifted = np.fft.fftshift(fft)
        return np.log(np.abs(fft_shifted))

def plot_frequency_analysis(im1, im2, sigma1, sigma2):
    hybrid = hybrid_image(im1, im2, sigma1, sigma2)
    low_freq = low_pass(im2, sigma2)
    high_freq = high_pass(im1, sigma1)
    # Convert to grayscale for frequency analysis
    gray1 = np.mean(im1, axis=2) if im1.ndim == 3 else im1
    gray2 = np.mean(im2, axis=2) if im2.ndim == 3 else im2
    gray_low = np.mean(low_freq, axis=2) if low_freq.ndim == 3 else low_freq
    gray_high = np.mean(high_freq, axis=2) if high_freq.ndim == 3 else high_freq
    gray_hybrid = np.mean(hybrid, axis=2) if hybrid.ndim == 3 else hybrid

    fft1 = compute_fft(gray1)
    fft2 = compute_fft(gray2)
    fft_low = compute_fft(gray_low)
    fft_high = compute_fft(gray_high)
    fft_hybrid = compute_fft(gray_hybrid)


    fig, axes = plt.subplots(2, 5, figsize=(20, 8))
    axes[0, 0].imshow(im1)
    axes[0, 0].set_title('Original Image 1\n(Nutmeg)')
    axes[0, 0].axis('off')
    axes[0, 1].imshow(im2)
    axes[0, 1].set_title('Original Image 2\n(Derek)')
    axes[0, 1].axis('off')


    axes[0, 2].imshow(low_freq)
    axes[0, 2].set_title(f'Low-pass Filtered\n( ={sigma2})')
    axes[0, 2].axis('off')
    axes[0, 3].imshow(high_freq + 0.5)
    axes[0, 3].set_title(f'High-pass Filtered\n( ={sigma1})')
    axes[0, 3].axis('off')
    axes[0, 4].imshow(hybrid)
    axes[0, 4].set_title('Hybrid Image')
    axes[0, 4].axis('off')


    axes[1, 0].imshow(fft1, cmap='viridis')
    axes[1, 0].set_title('FFT: Image 1')
    axes[1, 0].axis('off')
    axes[1, 1].imshow(fft2, cmap='viridis')
```

```
    axes[1, 1].set_title('FFT: Image 2')
    axes[1, 1].axis('off')
    axes[1, 2].imshow(fft_low, cmap='viridis')
    axes[1, 2].set_title('FFT: Low-pass')
    axes[1, 2].axis('off')
    axes[1, 3].imshow(fft_high, cmap='viridis')
    axes[1, 3].set_title('FFT: High-pass')
    axes[1, 3].axis('off')
    axes[1, 4].imshow(fft_hybrid, cmap='viridis')
    axes[1, 4].set_title('FFT: Hybrid')
    axes[1, 4].axis('off')

    plt.tight_layout()
    plt.suptitle(f'Hybrid Image Frequency Analysis ( 1={sigma1},  2={sigma2})',␣
↪fontsize=16, y=1.02)
    plt.show()

    return hybrid
```

[45]:
```
print("Frequency Analysis for Derek + Nutmeg:")
best_hybrid = plot_frequency_analysis(im1_aligned, im2_aligned, 13, 5)
```

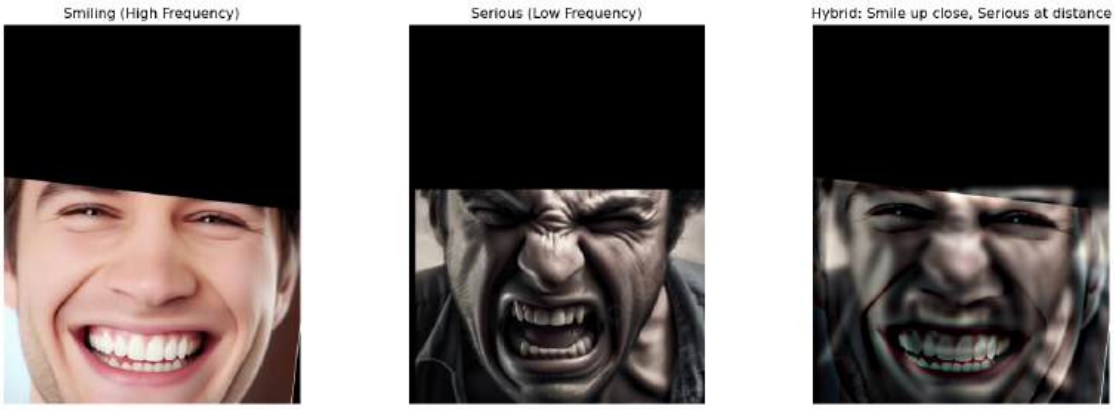
```
Frequency Analysis for Derek + Nutmeg:
Clipping input data to the valid range for imshow with RGB data ([0..1] for
floats or [0..255] for integers). Got range
[-0.010497815166470081..1.1577966359825345].
```



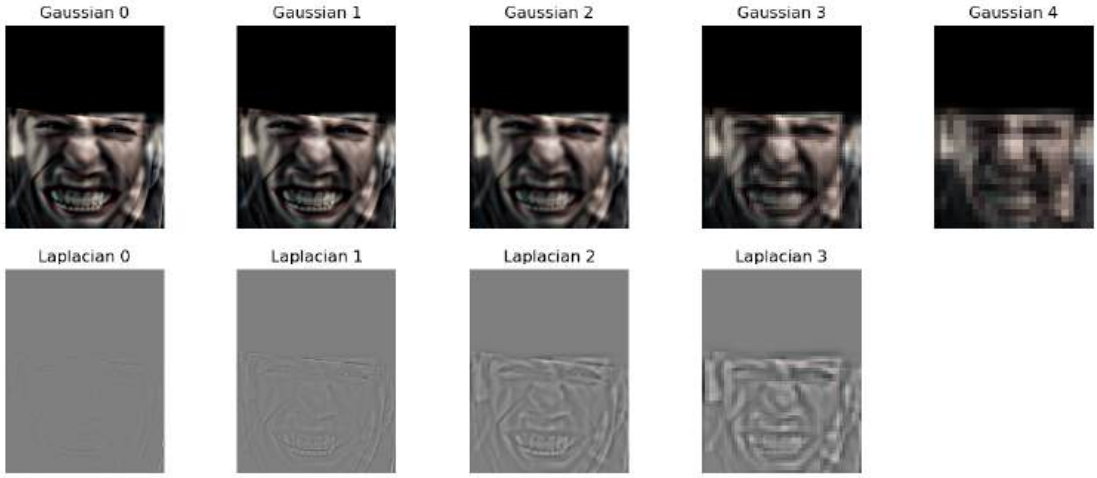
Try creating 2-3 hybrid images (change of expression, morph between different objects, change over time, etc.). Show the input image and hybrid result per example. (No need to show the intermediate results as in step 2.)

[18]:
```
def match_img_size(im1, im2):
    h1, w1, c1 = im1.shape
    h2, w2, c2 = im2.shape
    target_h = min(h1, h2)
    target_w = min(w1, w2)
    im1 = im1[(h1-target_h)//2:(h1-target_h)//2+target_h,
              (w1-target_w)//2:(w1-target_w)//2+target_w, :]
    im2 = im2[(h2-target_h)//2:(h2-target_h)//2+target_h,
              (w2-target_w)//2:(w2-target_w)//2+target_w, :]
    return im1, im2

# First example: Expression change (smiling to serious)
print("Example 1: Expression Change (Smiling  Raging)")

expr1 = plt.imread('smiling.jpg')/255.
expr2 = plt.imread('angry.jpg')/255.
%matplotlib qt
expr1_aligned, expr2_aligned = align_images(expr1, expr2)
%matplotlib inline

hybrid_expr = hybrid_image(expr1_aligned, expr2_aligned, sigma1=9, sigma2=4)


fig, axes = plt.subplots(1, 3, figsize=(15, 5))
axes[0].imshow(expr1_aligned)
axes[0].set_title('Smiling (High Frequency)')
axes[0].axis('off')

axes[1].imshow(expr2_aligned)
axes[1].set_title('Serious (Low Frequency)')
axes[1].axis('off')

axes[2].imshow(hybrid_expr)
axes[2].set_title('Hybrid: Smile up close, Serious at distance')
axes[2].axis('off')

plt.tight_layout()
plt.show()


pyramids(hybrid_expr, 5)
```

```
Example 1: Expression Change (Smiling  Raging)
Please select 2 points in each image for alignment.
```




```
Clipping input data to the valid range for imshow with RGB data ([0..1] for
floats or [0..255] for integers). Got range
[0.12254231381959163..1.0788717436511237].
```

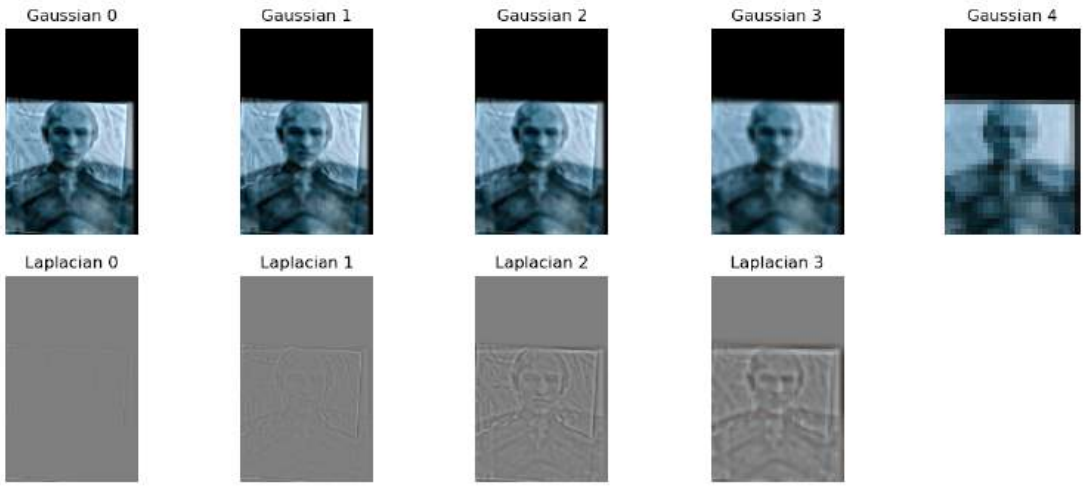


[19]:
```
print("Example 2: Object Morph (Human  Alien)")
alien_img = plt.imread('alien.jpg')/255.
human_img = plt.imread('human.jpg')/255.

%matplotlib qt
alien_aligned, human_aligned = align_images(human_img, alien_img)
%matplotlib inline

# Create hybrid - alien in high freq, human in low freq
hybrid_animals = hybrid_image(alien_aligned, human_aligned, sigma1=10, sigma2=4)
```

```
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
axes[0].imshow(alien_aligned)
axes[0].set_title('Alien (Low Frequency)')
axes[0].axis('off')

axes[1].imshow(human_aligned)
axes[1].set_title('Human (High Frequency)')
axes[1].axis('off')

axes[2].imshow(hybrid_animals)
axes[2].set_title('Hybrid: Human up close, Alien at distance')
axes[2].axis('off')

plt.tight_layout()
plt.show()

pyramids(hybrid_animals, 5)
```

```
Example 2: Object Morph (Human  Alien)
Please select 2 points in each image for alignment.
```


```
Clipping input data to the valid range for imshow with RGB data ([0..1] for
floats or [0..255] for integers). Got range
[0.07814514000627176..1.1146884551034666].
```



0.0.14 MULTI-RESOLUTION BLENDING AND THE ORAPLE JOURNEY

0.0.15 Overview

The goal of this part of the assignment is to blend two images seamlessly using a multi resolution blending as described in the 1983 paper by Burt and Adelson. An image spline is a smooth seam joining two image together by gently distorting them. Multiresolution blending computes a gentle seam between the two images seperately at each band of image frequencies, resulting in a much smoother seam.

We'll approach this section in two steps:

1. creating and visualizing the Gaussian and Laplacian stacks and
2. blending together images with the help of the completed stacks, and exploring creative outcomes

0.0.16 PART 2.3: GAUSSIAN AND LAPLACIAN STACKS

0.0.17 Overview

In this part you will implement Gaussian and Laplacian stacks, which are kind of like pyramids but without the downsampling. This will prepare you for the next step for Multi-resolution blending.

0.0.18 Details

1. Implement a Gaussian and a Laplacian stack. The different between a stack and a pyramid is that in each level of the pyramid the image is downsampled, so that the result gets smaller and smaller. In a stack the images are never downsampled so the results are all the same dimension as the original image, and can all be saved in one 3D matrix (if the original image was a grayscale image). To create the successive levels of the Gaussian Stack, just apply the Gaussian filter at each level, but do not subsample. In this way we will get a stack that behaves similarly to a pyramid that was downsampled to half its size at each level. If you would rather work with pyramids, you may implement pyramids other than stacks. However,

in any case, you are NOT allowed to use built-in pyramid functions like cv2.pyrDown() or skimage.transform.pyramid_gaussian() in this project. You must implement your stacks from scratch!

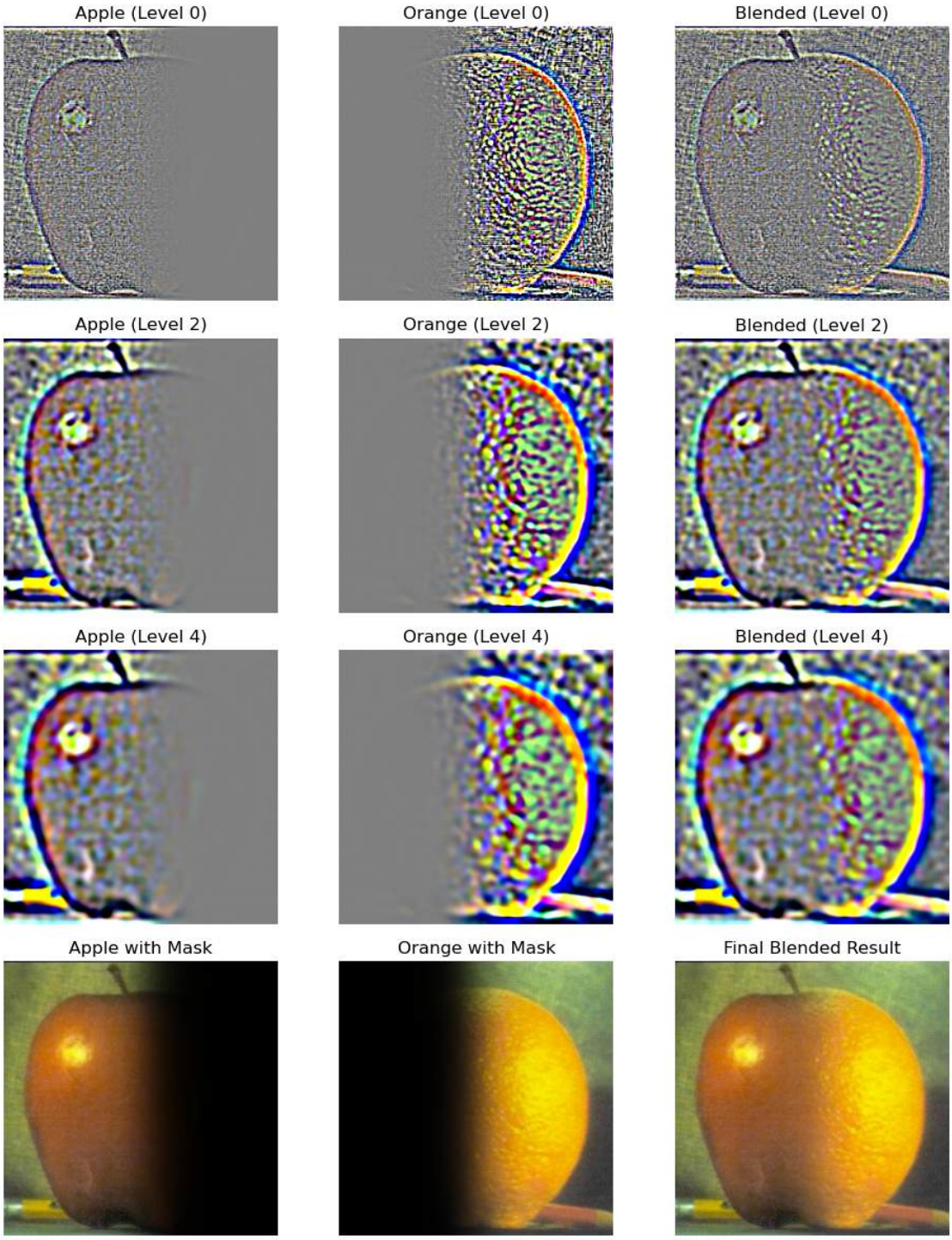
2. Apply your Gaussian and Laplacian stacks to the Oraple and recreate the outcomes of Figure 3.42 in Szelski (Ed 2) page 167, as you can see in the image above. Review the 1983 paper for more information.

[24]:
```
def build_gaussian_stack(img, num_levels=6, sigma=2):
    gaussian_stack = [img.astype(np.float32)]
    for _ in range(1, num_levels):
        blurred = cv2.GaussianBlur(gaussian_stack[-1], (0, 0), sigma)
        gaussian_stack.append(blurred)
    return gaussian_stack

def build_laplacian_stack(img, num_levels=6, sigma=2):
    gaussian_stack = build_gaussian_stack(img, num_levels, sigma)
    laplacian_stack = []
    for i in range(num_levels - 1):
        lap = gaussian_stack[i] - gaussian_stack[i+1]
        laplacian_stack.append(lap)
    laplacian_stack.append(gaussian_stack[-1])
    return laplacian_stack

def collapse_laplacian_stack(laplacian_stack):
    img_reconstructed = np.zeros_like(laplacian_stack[0])
    for layer in laplacian_stack:
        img_reconstructed += layer
    return np.clip(img_reconstructed, 0, 255).astype(np.uint8)


def blend_images_vertically(img1, img2, transition_width_ratio=1/20,␣
 ↪num_levels=6, sigma=2):
    h = min(img1.shape[0], img2.shape[0])
    w = min(img1.shape[1], img2.shape[1])
    img1_resized = cv2.resize(img1, (w, h))
    img2_resized = cv2.resize(img2, (w, h))
    mask = np.zeros((h, w), dtype=np.float32)
    transition_width = w * transition_width_ratio
    for x in range(w):
        mask[:, x] = 1.0 / (1.0 + np.exp((x - w//2) / transition_width))
    mask = np.stack([mask, mask, mask], axis=2)
    img1_lap = build_laplacian_stack(img1_resized, num_levels, sigma)
    img2_lap = build_laplacian_stack(img2_resized, num_levels, sigma)
    mask_gauss = build_gaussian_stack(mask, num_levels, sigma)
    blended_stack = []
    for i in range(num_levels):
```

```
        blended = mask_gauss[i] * img1_lap[i] + (1 - mask_gauss[i]) *␣
↪img2_lap[i]
        blended_stack.append(blended)
    blended_image = collapse_laplacian_stack(blended_stack)
    return blended_image, mask


def norm_img(img):
        max_val = max(np.abs(img.min()), np.abs(img.max()))
        if max_val > 0:
            img_disp = 128 + (img / max_val) * 1024
        else:
            img_disp = np.zeros_like(img) + 128
        return np.clip(img_disp, 0, 255).astype(np.uint8)
```

[26]:
```
apple = cv2.imread("apple.jpeg")
orange = cv2.imread("orange.jpeg")
h = min(apple.shape[0], orange.shape[0])
w = min(apple.shape[1], orange.shape[1])
apple = cv2.resize(apple, (w, h))
orange = cv2.resize(orange, (w, h))

mask = np.zeros((h, w), dtype=np.float32)
transition_width = w / 20
for x in range(w):
    mask[:, x] = 1.0 / (1.0 + np.exp((x - w//2) / transition_width))
mask = np.stack([mask, mask, mask], axis=2)

num_levels = 6
apple_lap = build_laplacian_stack(apple, num_levels)
orange_lap = build_laplacian_stack(orange, num_levels)
mask_gauss = build_gaussian_stack(mask, num_levels)
blended_stack = []
for i in range(num_levels):
    blended = mask_gauss[i] * apple_lap[i] + (1 - mask_gauss[i]) * orange_lap[i]
    blended_stack.append(blended)
oraple = collapse_laplacian_stack(blended_stack)

plt.figure(figsize=(10, 12))
levels_to_show = [0, 2, 4]
row = 0
for i, level in enumerate(levels_to_show):
    Hl = mask_gauss[level]
    Hr = 1 - Hl
    apple_contrib = Hl * apple_lap[level]
    orange_contrib = Hr * orange_lap[level]
    blended_contrib = blended_stack[level]
```

```
    plt.subplot(4, 3, row*3 + 1)
    plt.imshow(cv2.cvtColor(norm_img(apple_contrib), cv2.COLOR_BGR2RGB))
    plt.title(f"Apple (Level {level})")
    plt.axis("off")
    plt.subplot(4, 3, row*3 + 2)
    plt.imshow(cv2.cvtColor(norm_img(orange_contrib), cv2.COLOR_BGR2RGB))
    plt.title(f"Orange (Level {level})")
    plt.axis("off")
    plt.subplot(4, 3, row*3 + 3)
    plt.imshow(cv2.cvtColor(norm_img(blended_contrib), cv2.COLOR_BGR2RGB))
    plt.title(f"Blended (Level {level})")
    plt.axis("off")
    row += 1

apple_with_mask = apple.astype(np.float32) * mask_gauss[0]
apple_with_mask = np.clip(apple_with_mask, 0, 255).astype(np.uint8)
plt.subplot(4, 3, 10)
plt.imshow(cv2.cvtColor(apple_with_mask, cv2.COLOR_BGR2RGB))
plt.title("Apple with Mask")
plt.axis("off")
orange_with_mask = orange.astype(np.float32) * (1 - mask_gauss[0])
orange_with_mask = np.clip(orange_with_mask, 0, 255).astype(np.uint8)
plt.subplot(4, 3, 11)
plt.imshow(cv2.cvtColor(orange_with_mask, cv2.COLOR_BGR2RGB))
plt.title("Orange with Mask")
plt.axis("off")
plt.subplot(4, 3, 12)
plt.imshow(cv2.cvtColor(oraple, cv2.COLOR_BGR2RGB))
plt.title("Final Blended Result")
plt.axis("off")
plt.tight_layout()
plt.show()
```


Apple (Level 0)
Orange (Level 0)
Blended (Level 0)
Apple (Level 2)
Orange (Level 2)
Blended (Level 2)
Apple (Level 4)
Orange (Level 4)
Blended (Level 4)
Apple with Mask
Orange with Mask
Final Blended Result

0.0.19 Part 2.4: Multiresolution Blending (a.k.a. the oraple!)

0.0.20 Overview

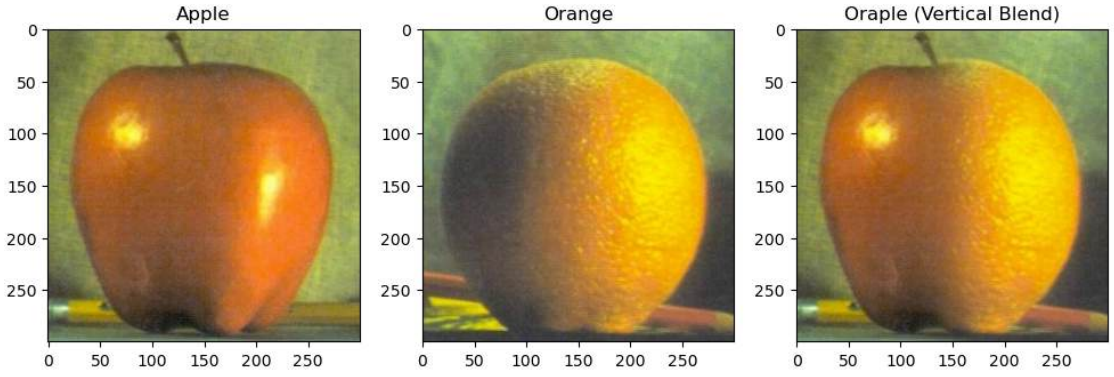
Review the 1983 paper by Burt and Adelson, if you haven't! This will provide you with the context to continue. In this part, we'll focus on actually blending two images together.

0.0.21 Details

Here, we have included the two sample images from the paper (of an apple and an orange).

1. First, you'll need to get a few pairs of images that you want blend together with a vertical or horizontal seam. You can use the sample images for debugging, but you should use your own images in your results. Then you will need to write some code in order to use your Gaussian and Laplacian stacks from part 2 in order to blend the images together. Since we are using stacks instead of pyramids like in the paper, the algorithm described on page 226 will not work as-is. If you try it out, you will find that you end up with a very clear seam between the apple and the orange since in the pyramid case the downsampling/blurring/upsampling hoopla ends up blurring the abrupt seam proposed in this algorithm. Instead, you should always use a mask as is proposed in the algorithm on page 230, and remember to create a Gaussian stack for your mask image as well as for the two input images. The Gaussian blurring of the mask in the pyramid will smooth out the transition between the two images. For the vertical or horizontal seam, your mask will simply be a step function of the same size as the original images.

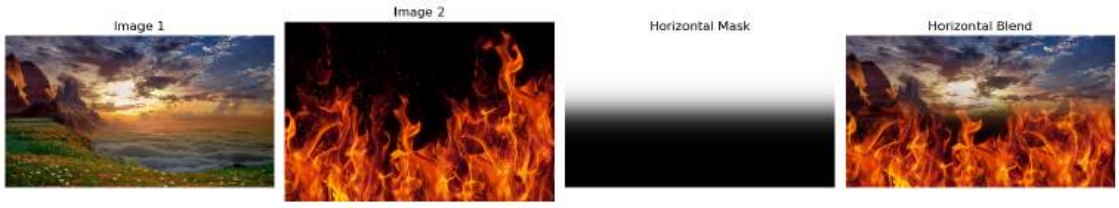
[27]:
```
oraple, mask = blend_images_vertically(apple, orange, transition_width_ratio=1/
↪20)

plt.figure(figsize=(12,6))
plt.subplot(1,3,1); plt.imshow(cv2.cvtColor(apple, cv2.COLOR_BGR2RGB)); plt.
↪title("Apple")
plt.subplot(1,3,2); plt.imshow(cv2.cvtColor(orange, cv2.COLOR_BGR2RGB)); plt.
↪title("Orange")
plt.subplot(1,3,3); plt.imshow(cv2.cvtColor(oraple, cv2.COLOR_BGR2RGB)); plt.
↪title("Oraple (Vertical Blend)")
plt.show()
```



[28]:
```
def blend_images_horizontally(img1, img2, transition_width_ratio=1/20):
    h = min(img1.shape[0], img2.shape[0])
    w = min(img1.shape[1], img2.shape[1])
    img1_resized = cv2.resize(img1, (w, h))
    img2_resized = cv2.resize(img2, (w, h))

    mask_horizontal = np.zeros((h, w), dtype=np.float32)
    transition_width = h * transition_width_ratio
    for y in range(h):
        mask_horizontal[y, :] = 1.0 / (1.0 + np.exp((y - h//2) /␣
 ↪transition_width))
    mask_horizontal = np.stack([mask_horizontal, mask_horizontal,␣
 ↪mask_horizontal], axis=2)

    num_levels = 6
    img1_lap = build_laplacian_stack(img1_resized, num_levels)
    img2_lap = build_laplacian_stack(img2_resized, num_levels)
    mask_gauss_horizontal = build_gaussian_stack(mask_horizontal, num_levels)
    blended_stack_horizontal = []
    for i in range(num_levels):
        blended = mask_gauss_horizontal[i] * img1_lap[i] + (1 -␣
 ↪mask_gauss_horizontal[i]) * img2_lap[i]
        blended_stack_horizontal.append(blended)
    blended_image = collapse_laplacian_stack(blended_stack_horizontal)

    return blended_image, mask_horizontal
```

[56]:
```
image1 = cv2.imread("heaven.jpg")
image2 = cv2.imread("hell.jpg")
blended_result, mask_used = blend_images_horizontally(image1, image2)

plt.figure(figsize=(15, 5))
plt.subplot(1, 4, 1)
plt.imshow(cv2.cvtColor(image1, cv2.COLOR_BGR2RGB))
plt.title("Image 1")
plt.axis('off')

plt.subplot(1, 4, 2)
plt.imshow(cv2.cvtColor(image2, cv2.COLOR_BGR2RGB))
plt.title("Image 2")
plt.axis('off')

plt.subplot(1, 4, 3)
plt.imshow(mask_used[:, :, 0], cmap='gray')
plt.title("Horizontal Mask")
```

```
plt.axis('off')

plt.subplot(1, 4, 4)
plt.imshow(cv2.cvtColor(blended_result, cv2.COLOR_BGR2RGB))
plt.title("Horizontal Blend")
plt.axis('off')

plt.tight_layout()
plt.show()
```



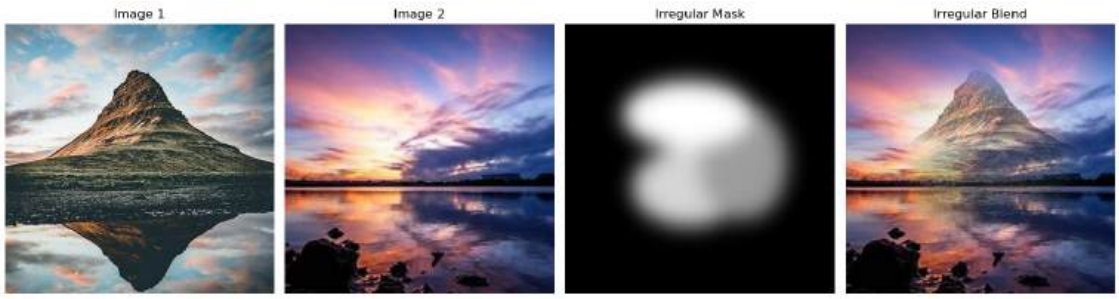
2. Now that you've made yourself an oraple (a.k.a your vertical or horizontal seam is nicely working), pick two pairs of images to blend together with an irregular mask, as is demonstrated in figure 8 in the paper.

[30]:
```
def create_irregular_mask(shape, num_shapes=3):
    h, w = shape[:2]
    mask = np.zeros((h, w), dtype=np.float32)
    for _ in range(num_shapes):
        center_x = np.random.randint(w//4, 3*w//4)
        center_y = np.random.randint(h//4, 3*h//4)
        radius_x = np.random.randint(w//8, w//4)
        radius_y = np.random.randint(h//8, h//4)
        y, x = np.ogrid[:h, :w]
        ellipse = ((x - center_x) / radius_x)**2 + ((y - center_y) /␣
↪radius_y)**2 <= 1

        mask[ellipse] = np.random.uniform(0.3, 1.0)
    mask = cv2.GaussianBlur(mask, (0, 0), 10)
    return np.stack([mask, mask, mask], axis=2)


def blend_with_irregular_mask(img1, img2, num_shapes=3, num_levels=6, sigma=2):
    h = min(img1.shape[0], img2.shape[0])
    w = min(img1.shape[1], img2.shape[1])
    img1_resized = cv2.resize(img1, (w, h))
    img2_resized = cv2.resize(img2, (w, h))
    mask = create_irregular_mask((h, w), num_shapes)
    img1_lap = build_laplacian_stack(img1_resized, num_levels, sigma)
```

```
    img2_lap = build_laplacian_stack(img2_resized, num_levels, sigma)
    mask_gauss = build_gaussian_stack(mask, num_levels, sigma)
    blended_stack = []
    for i in range(num_levels):
        blended = mask_gauss[i] * img1_lap[i] + (1 - mask_gauss[i]) *␣
 ↪img2_lap[i]
        blended_stack.append(blended)
    blended_image = collapse_laplacian_stack(blended_stack)
    return blended_image, mask
```

[103]:
```
image2 = cv2.imread("forests.jpg")
image1 = cv2.imread("forest.jpg")
blended_result, mask_used = blend_with_irregular_mask(image1, image2,␣
 ↪num_shapes=5)

plt.figure(figsize=(15, 5))
plt.subplot(1, 4, 1)
plt.imshow(cv2.cvtColor(image1, cv2.COLOR_BGR2RGB))
plt.title("Image 1")
plt.axis('off')

plt.subplot(1, 4, 2)
plt.imshow(cv2.cvtColor(image2, cv2.COLOR_BGR2RGB))
plt.title("Image 2")
plt.axis('off')

plt.subplot(1, 4, 3)
plt.imshow(mask_used[:, :, 0], cmap='gray')
plt.title("Irregular Mask")
plt.axis('off')

plt.subplot(1, 4, 4)
plt.imshow(cv2.cvtColor(blended_result, cv2.COLOR_BGR2RGB))
plt.title("Irregular Blend")
plt.axis('off')

plt.tight_layout()
plt.show()
```

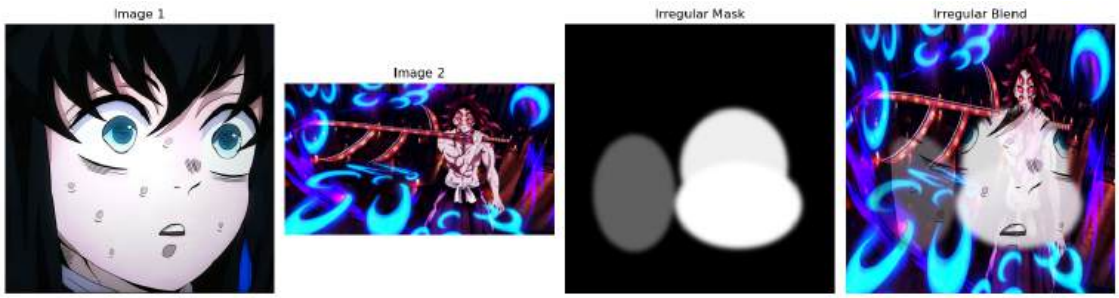
[51]:
```
image1 = cv2.imread("mountain.jpg")
image2 = cv2.imread("sunset.jpg")
blended_result, mask_used = blend_with_irregular_mask(image1, image2)

plt.figure(figsize=(15, 5))
plt.subplot(1, 4, 1)
plt.imshow(cv2.cvtColor(image1, cv2.COLOR_BGR2RGB))
plt.title("Image 1")
plt.axis('off')

plt.subplot(1, 4, 2)
plt.imshow(cv2.cvtColor(image2, cv2.COLOR_BGR2RGB))
plt.title("Image 2")
plt.axis('off')

plt.subplot(1, 4, 3)
plt.imshow(mask_used[:, :, 0], cmap='gray')
plt.title("Irregular Mask")
plt.axis('off')

plt.subplot(1, 4, 4)
plt.imshow(cv2.cvtColor(blended_result, cv2.COLOR_BGR2RGB))
plt.title("Irregular Blend")
plt.axis('off')

plt.tight_layout()
plt.show()
```



3. Blend together some crazy ideas of your own!

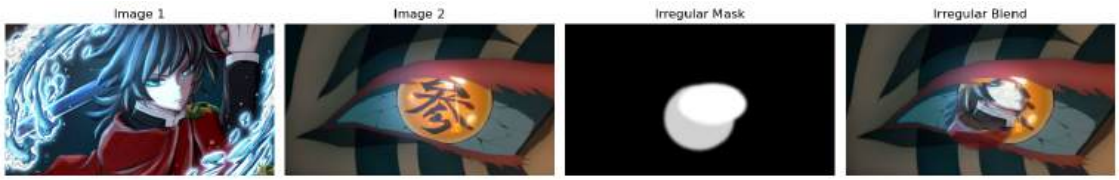
[50]:
```
image1 = cv2.imread("muichiro.jpg")
image2 = cv2.imread("kokushibo.jpg")
blended_result, mask_used = blend_with_irregular_mask(image1, image2)


plt.figure(figsize=(15, 5))
```

```
plt.subplot(1, 4, 1)
plt.imshow(cv2.cvtColor(image1, cv2.COLOR_BGR2RGB))
plt.title("Image 1")
plt.axis('off')

plt.subplot(1, 4, 2)
plt.imshow(cv2.cvtColor(image2, cv2.COLOR_BGR2RGB))
plt.title("Image 2")
plt.axis('off')

plt.subplot(1, 4, 3)
plt.imshow(mask_used[:, :, 0], cmap='gray')
plt.title("Irregular Mask")
plt.axis('off')

plt.subplot(1, 4, 4)
plt.imshow(cv2.cvtColor(blended_result, cv2.COLOR_BGR2RGB))
plt.title("Irregular Blend")
plt.axis('off')

plt.tight_layout()
plt.show()
```



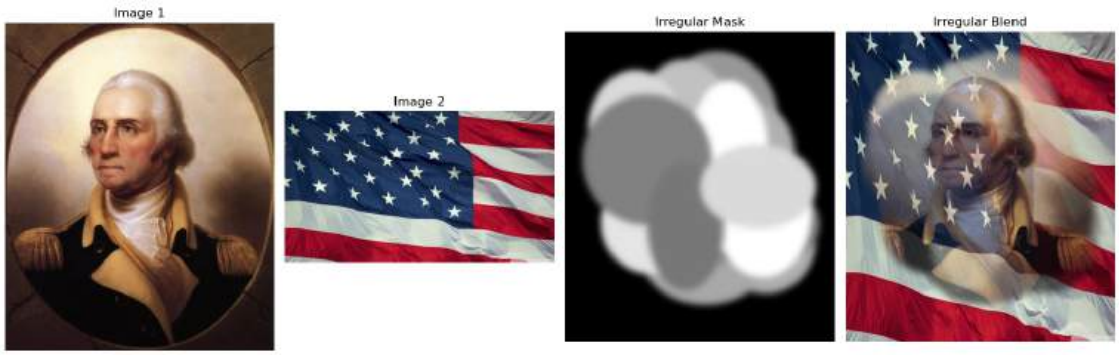
[98]:
```
image2 = cv2.imread("akaza.png")
image1 = cv2.imread("giyu.jpg")
blended_result, mask_used = blend_with_irregular_mask(image1, image2,␣
 ↪num_shapes=2)


plt.figure(figsize=(15, 5))
plt.subplot(1, 4, 1)
plt.imshow(cv2.cvtColor(image1, cv2.COLOR_BGR2RGB))
plt.title("Image 1")
plt.axis('off')
```

```
plt.subplot(1, 4, 2)
plt.imshow(cv2.cvtColor(image2, cv2.COLOR_BGR2RGB))
plt.title("Image 2")
plt.axis('off')

plt.subplot(1, 4, 3)
plt.imshow(mask_used[:, :, 0], cmap='gray')
plt.title("Irregular Mask")
plt.axis('off')

plt.subplot(1, 4, 4)
plt.imshow(cv2.cvtColor(blended_result, cv2.COLOR_BGR2RGB))
plt.title("Irregular Blend")
plt.axis('off')

plt.tight_layout()
plt.show()
```



[113]:
```
image2 = cv2.imread("america.jpg")
image1 = cv2.imread("george.jpg")
blended_result, mask_used = blend_with_irregular_mask(image1, image2,␣
 ↪num_shapes=20)


plt.figure(figsize=(15, 5))
plt.subplot(1, 4, 1)
plt.imshow(cv2.cvtColor(image1, cv2.COLOR_BGR2RGB))
plt.title("Image 1")
plt.axis('off')

plt.subplot(1, 4, 2)
plt.imshow(cv2.cvtColor(image2, cv2.COLOR_BGR2RGB))
plt.title("Image 2")
plt.axis('off')

plt.subplot(1, 4, 3)
plt.imshow(mask_used[:, :, 0], cmap='gray')
plt.title("Irregular Mask")
plt.axis('off')
```

```
plt.subplot(1, 4, 4)
plt.imshow(cv2.cvtColor(blended_result, cv2.COLOR_BGR2RGB))
plt.title("Irregular Blend")
plt.axis('off')

plt.tight_layout()
plt.show()
```

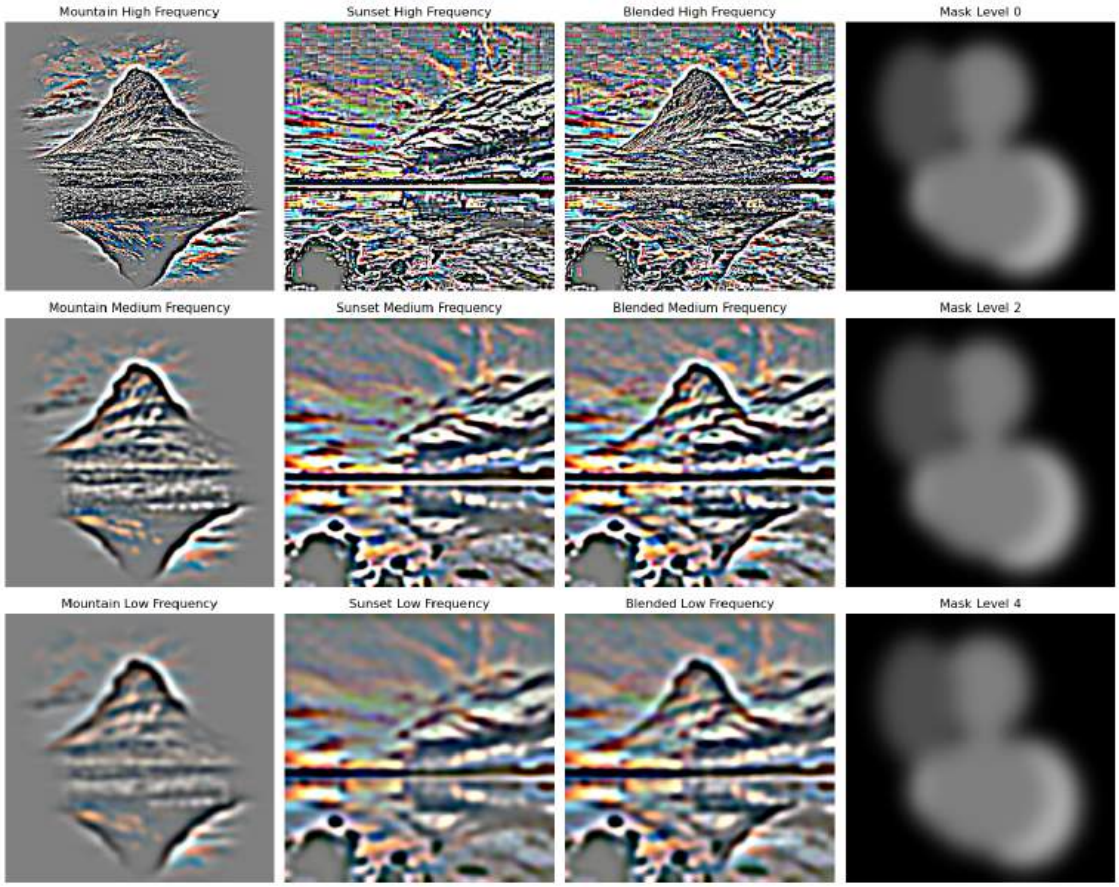


4. Illustrate the process by applying your Laplacian stack and displaying it for your favorite result and the masked input images that created it. This should look similar to Figure 10 in the paper.

[132]:
```
mountain = cv2.imread("mountain.jpg")
sunset = cv2.imread("sunset.jpg")
h = min(mountain.shape[0], sunset.shape[0])
w = min(mountain.shape[1], sunset.shape[1])
mountain_resized = cv2.resize(mountain, (w, h))
sunset_resized = cv2.resize(sunset, (w, h))


irregular_mask = create_irregular_mask((h, w), num_shapes=5)
num_levels = 6
mountain_lap = build_laplacian_stack(mountain_resized, num_levels)
sunset_lap = build_laplacian_stack(sunset_resized, num_levels)
mask_gauss = build_gaussian_stack(irregular_mask, num_levels)

blended_stack = []
for i in range(num_levels):
    blended = mask_gauss[i] * mountain_lap[i] + (1 - mask_gauss[i]) *␣
 ↪sunset_lap[i]
    blended_stack.append(blended)
```

```
mountain_sunset_blend = collapse_laplacian_stack(blended_stack)


plt.figure(figsize=(15, 12))
levels_to_show = [0, 2, 4]
frequency_labels = ["High Frequency", "Medium Frequency", "Low Frequency"]

for i, level in enumerate(levels_to_show):
    mountain_component = mask_gauss[level] * mountain_lap[level]
    plt.subplot(3, 4, i*4 + 1)
    mountain_display = 128 + (mountain_component - np.mean(mountain_component))␣
 ↪* 64
    mountain_display = np.clip(mountain_display, 0, 255).astype(np.uint8)
    plt.imshow(cv2.cvtColor(mountain_display, cv2.COLOR_BGR2RGB))
    plt.title(f"Mountain {frequency_labels[i]}")
    plt.axis('off')
    sunset_component = (1 - mask_gauss[level]) * sunset_lap[level]
    plt.subplot(3, 4, i*4 + 2)
    sunset_display = 128 + (sunset_component - np.mean(sunset_component)) * 64
    sunset_display = np.clip(sunset_display, 0, 255).astype(np.uint8)
    plt.imshow(cv2.cvtColor(sunset_display, cv2.COLOR_BGR2RGB))
    plt.title(f"Sunset {frequency_labels[i]}")
    plt.axis('off')
    blended_component = blended_stack[level]
    plt.subplot(3, 4, i*4 + 3)
    blended_display = 128 + (blended_component - np.mean(blended_component)) *␣
 ↪64
    blended_display = np.clip(blended_display, 0, 255).astype(np.uint8)
    plt.imshow(cv2.cvtColor(blended_display, cv2.COLOR_BGR2RGB))
    plt.title(f"Blended {frequency_labels[i]}")
    plt.axis('off')

    plt.subplot(3, 4, i*4 + 4)
    plt.imshow(mask_gauss[level][:, :, 0], cmap='gray', vmin=0, vmax=1)
    plt.title(f"Mask Level {level}")
    plt.axis('off')
plt.tight_layout()
plt.show()


plt.figure(figsize=(15, 5))
plt.subplot(1, 4, 1)
plt.imshow(cv2.cvtColor(mountain_resized, cv2.COLOR_BGR2RGB))
plt.title("Mountain")
plt.axis('off')
plt.subplot(1, 4, 2)
plt.imshow(cv2.cvtColor(sunset_resized, cv2.COLOR_BGR2RGB))
```

```
plt.title("Sunset")
plt.axis('off')
plt.subplot(1, 4, 3)
plt.imshow(irregular_mask[:, :, 0], cmap='gray')
plt.title("Irregular Mask")
plt.axis('off')
plt.subplot(1, 4, 4)
plt.imshow(cv2.cvtColor(mountain_sunset_blend, cv2.COLOR_BGR2RGB))
plt.title("Mountain Sunset Blend")
plt.axis('off')
plt.tight_layout()
plt.show()
```




Mountain
Sunset
Irregular Mask
Mountain Sunset Blend

[ ]: